

Deep Learning for Text Analytics



Network Architectures for Sequential Data – Recurrent Neural Networks II

Stefan Lessmann [stefan.lessmann@hu-berlin.de]



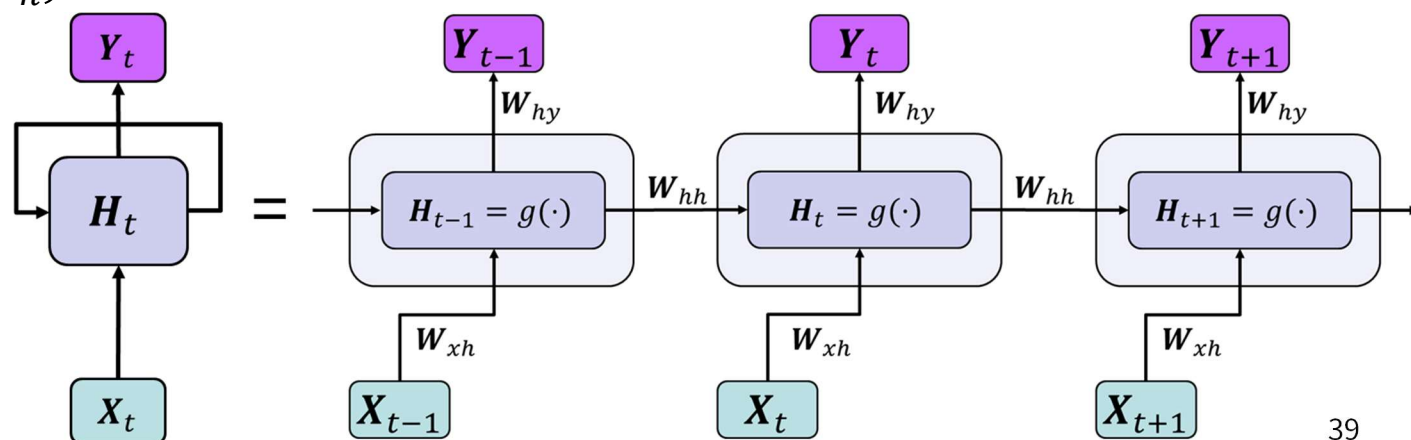
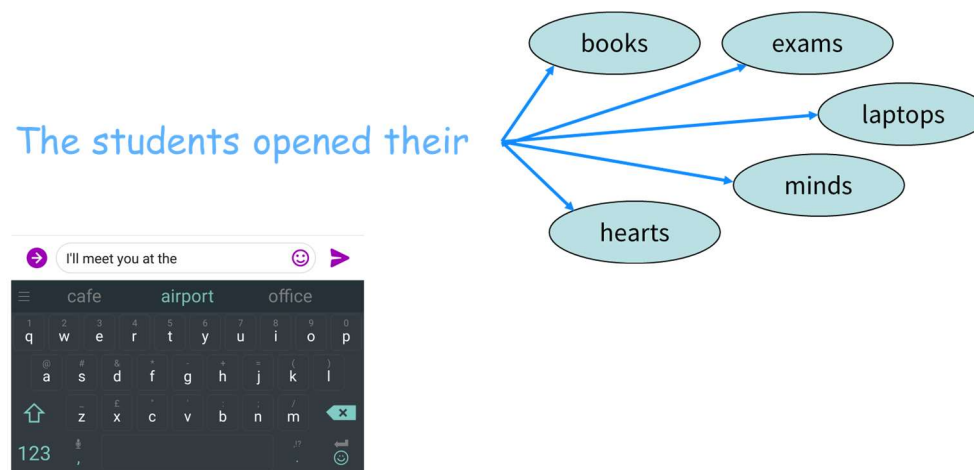
RECAP: LMs, State-Space Models, and RNNs

■ Language models

- Predict next token in a text sequence
- $p(x_{t+1} | x_t, x_{t-1}, \dots, x_1)$
- Subcomponent in many NLP tasks

■ Recurrent cells and networks

- Latent variable auto-regressive model
- $H_t = g(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$
- $\hat{Y}_t = H_t W_{hy} + b_y$
- General framework for processing sequential data
- Well-suited for language modeling and text analytics



RECAP: Generating Text with an RNN-LM

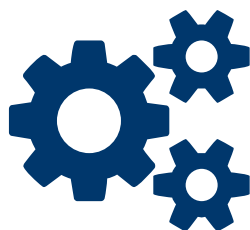
Use 1-step forecast recursively

- Input sequence with predefined length to accumulate hidden state
- Apply output-weight matrix, to obtain raw output scores
- Softmax raw scores to obtain probability distribution
- Select most probable next token as prediction
- Shift input window and repeat

Corpus



Sampling



started as early as eight in

in the morning <comma> talking about

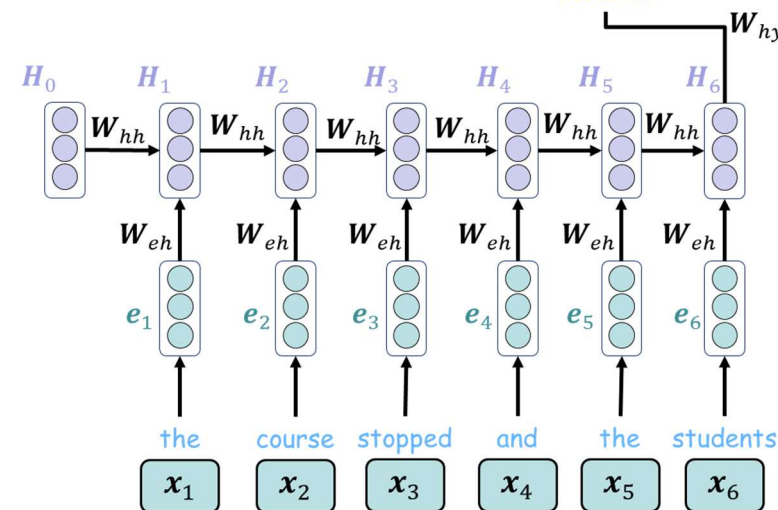
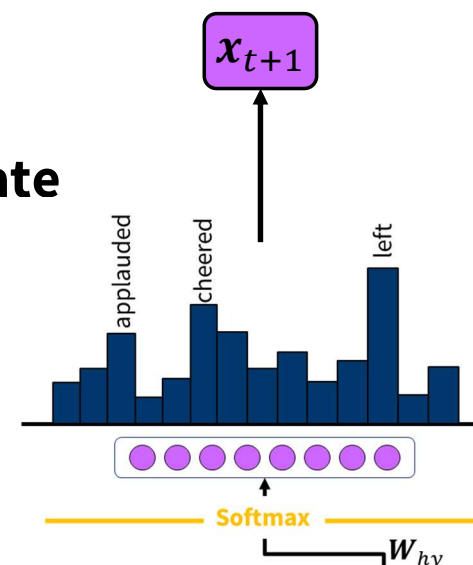
about RNNs <dot> After a daunting

daunting two hours it eventually turned

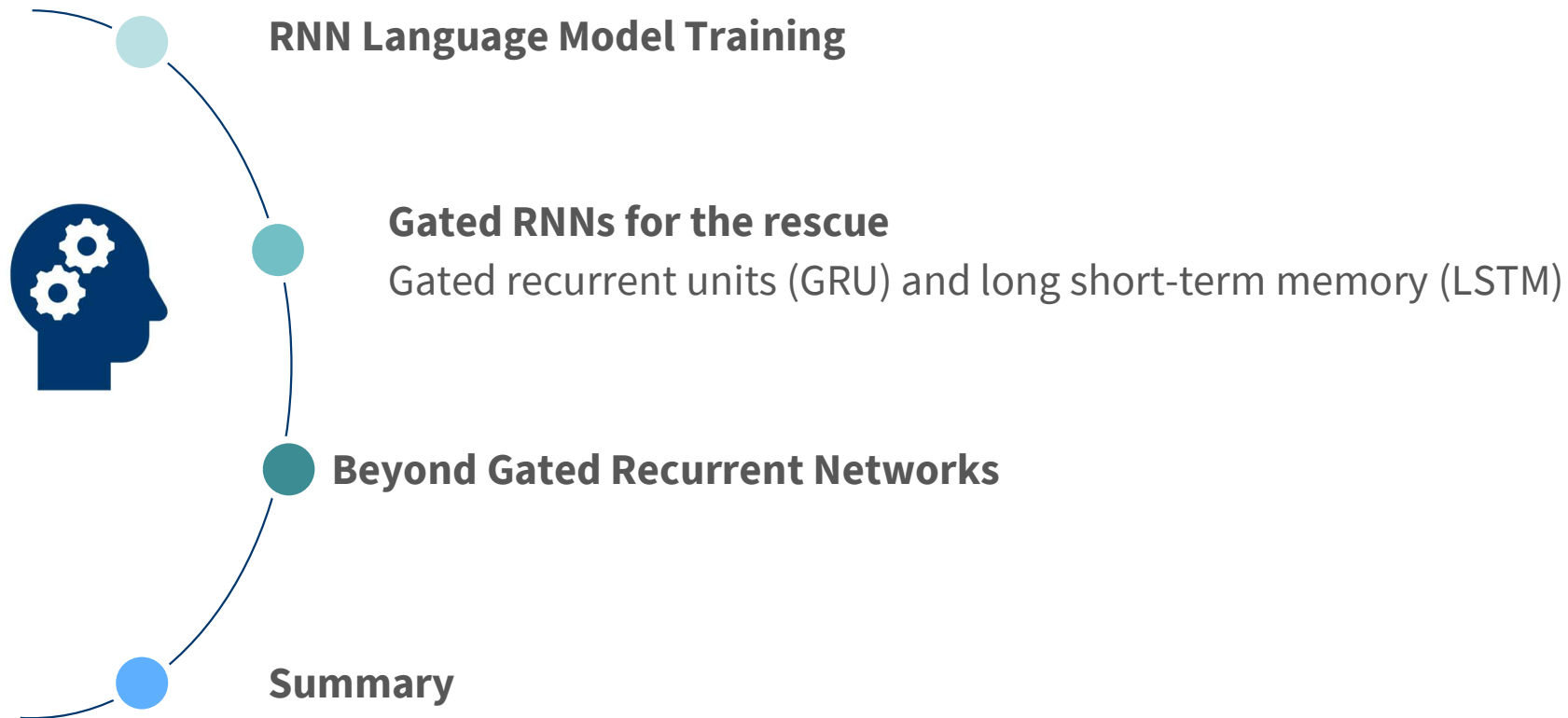
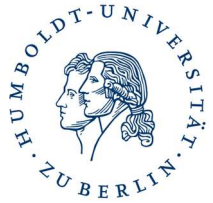
eventually turned ten and the course

the course stopped and the students

students closed their books and left



Agenda





RNN Language Model Training

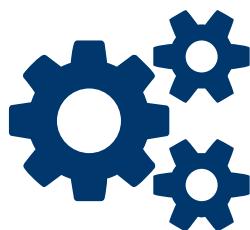
RNN Language Model Training – Main Ideas

- Gather (large) corpus of text
- Set sequence length and parse text
- Feed input sequence to RNN (with random weights)
- Compute output distribution for every step
- Assess loss using cross-entropy loss function
- Update weights using back-propagation

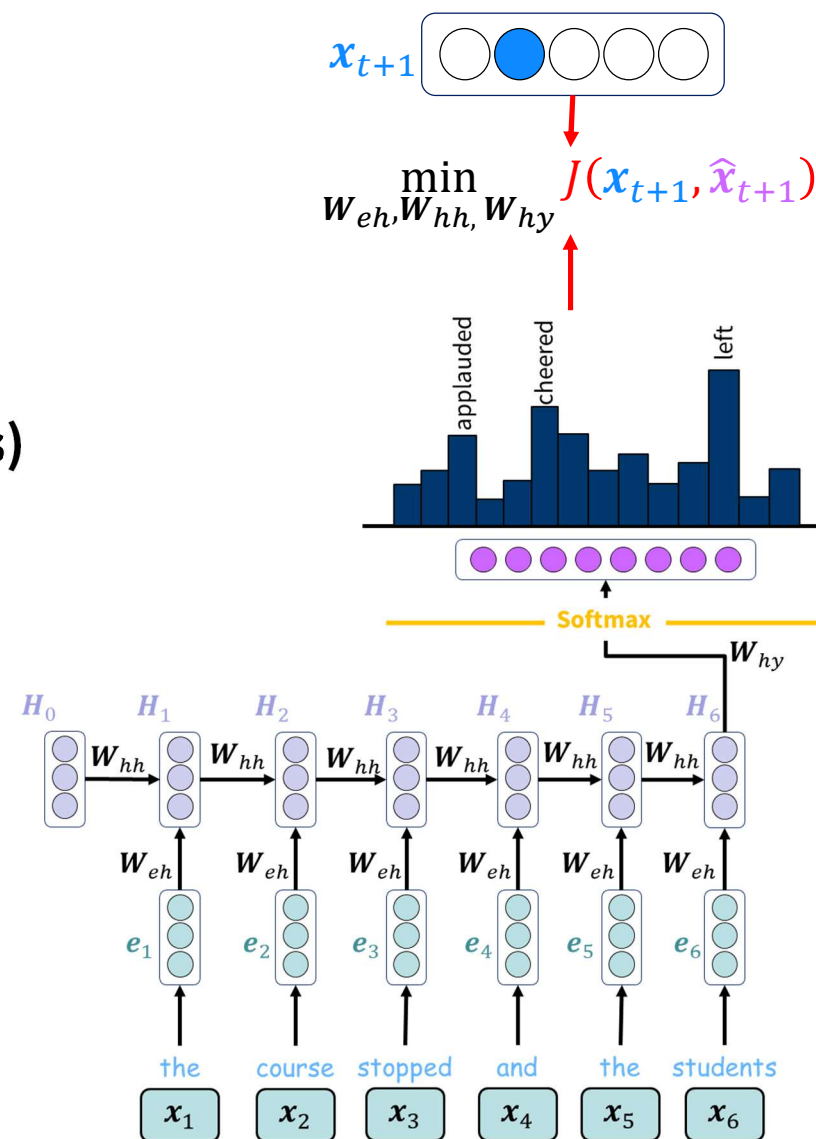
Corpus



Sampling



started as early as eight in
 in the morning <comma> talking about
 about RNNs <dot> After a daunting
 daunting two hours it eventually turned
 eventually turned ten and the course
 the course stopped and the students
 students closed their books and left



Cross-Entropy Loss Function for Language Modeling

Loss equals log-probability of the next token

The course stopped and the students closed

RNN LM

$$\hat{x}_{t+1} \sim p(x_{t+1} | x_t, x_{t-1}, \dots, x_1)$$

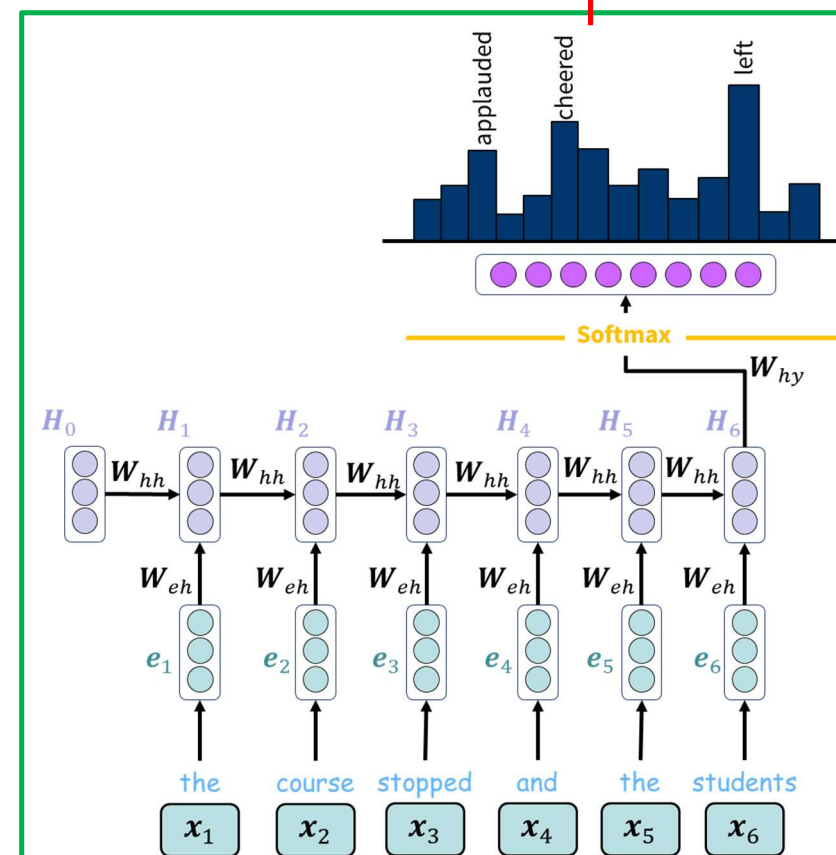
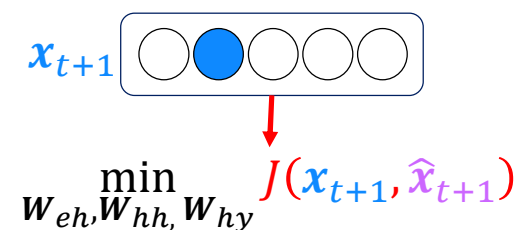
$$\sim p(\text{closed} | \text{students, the, ... , the})$$

CE Loss

$$J(W) = -x_{t+1} \cdot \log(\hat{x}_{t+1})$$

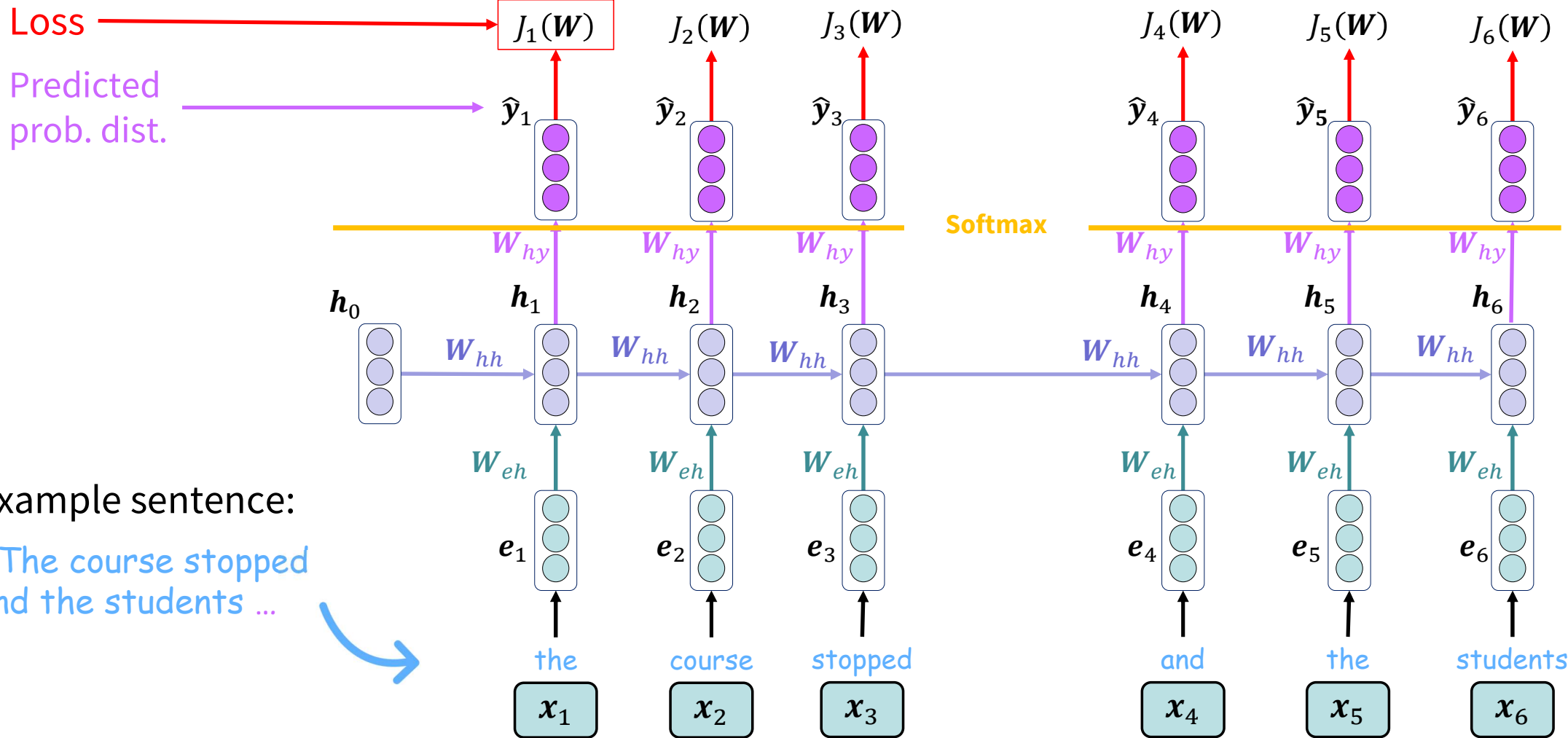
with $x_t \in \{0,1\}^{|V|} \forall t$,

$$\hat{x}_{t+1} = f_W(x_{t-p+1:t}) \in (0,1)^{|V|} \forall t$$

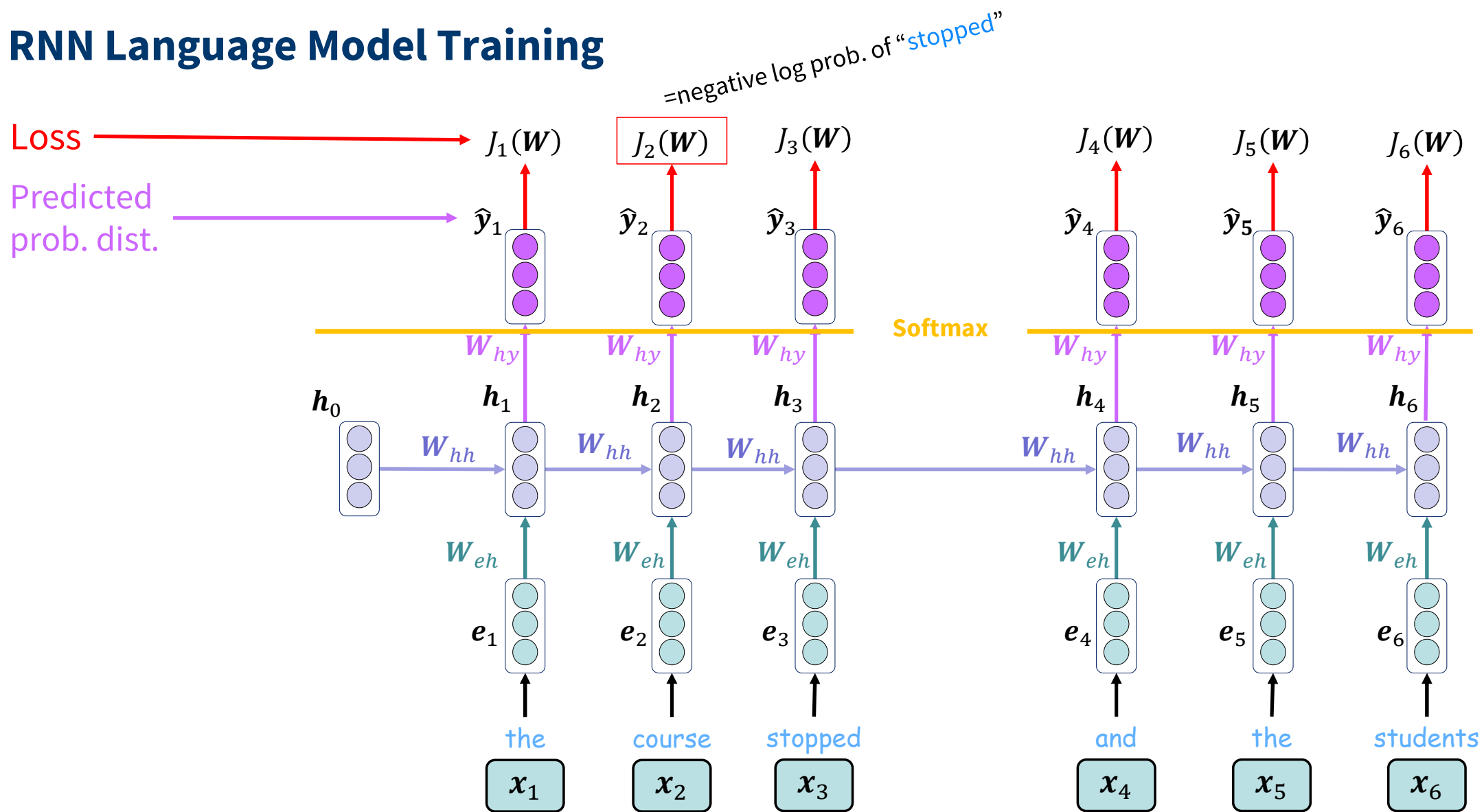


Compute Loss Each Step

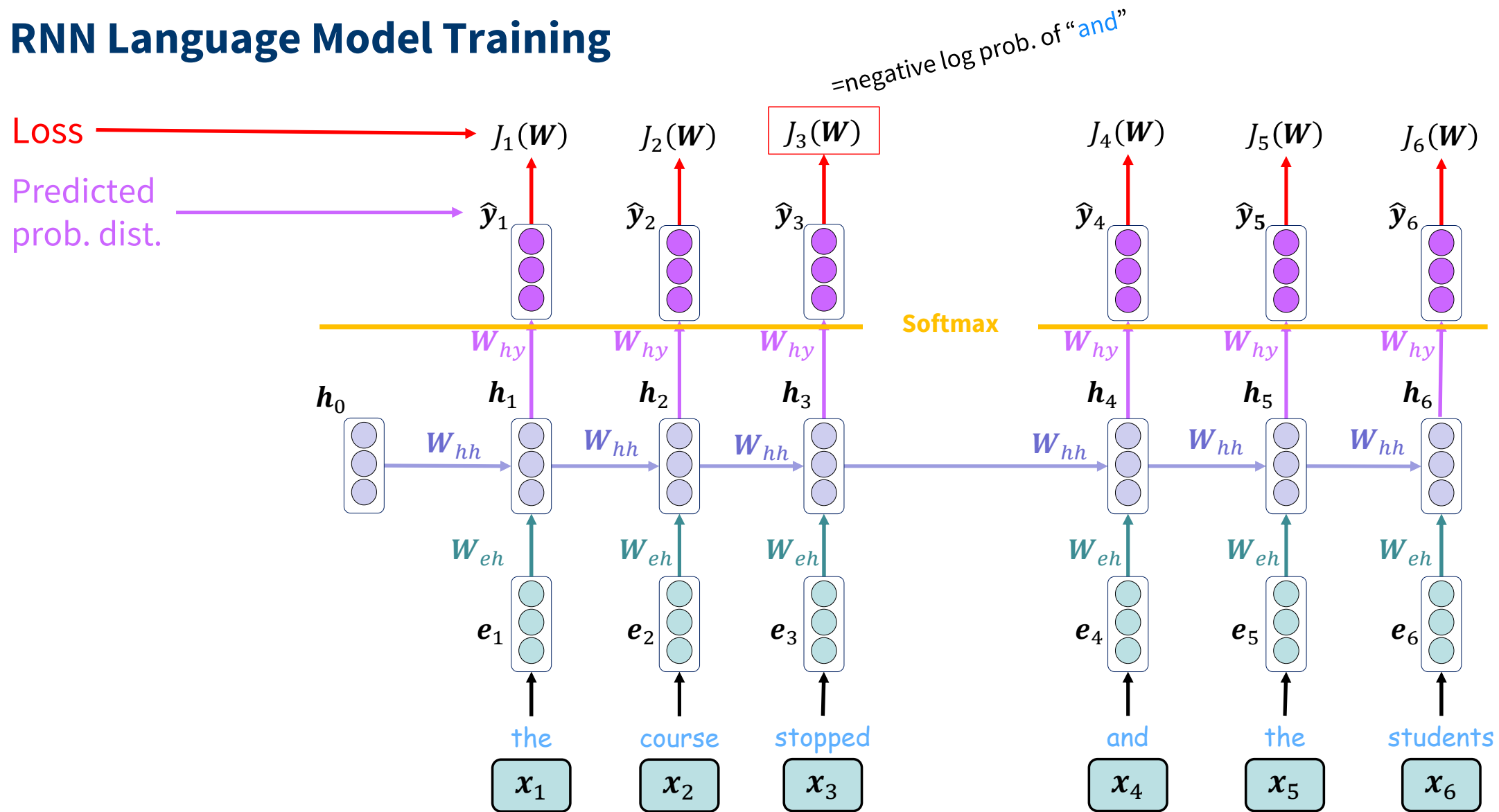
=negative log prob. of "course"



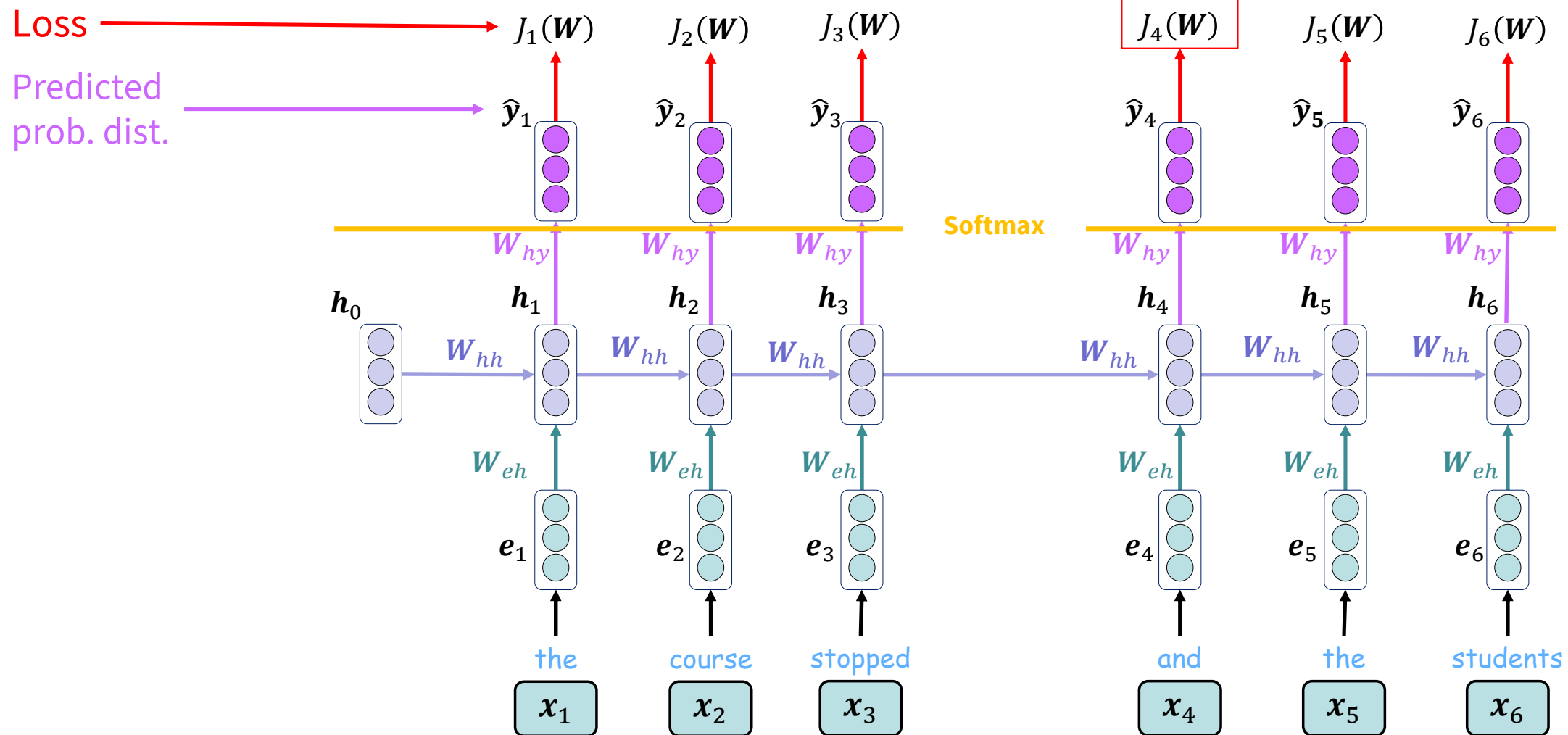
RNN Language Model Training



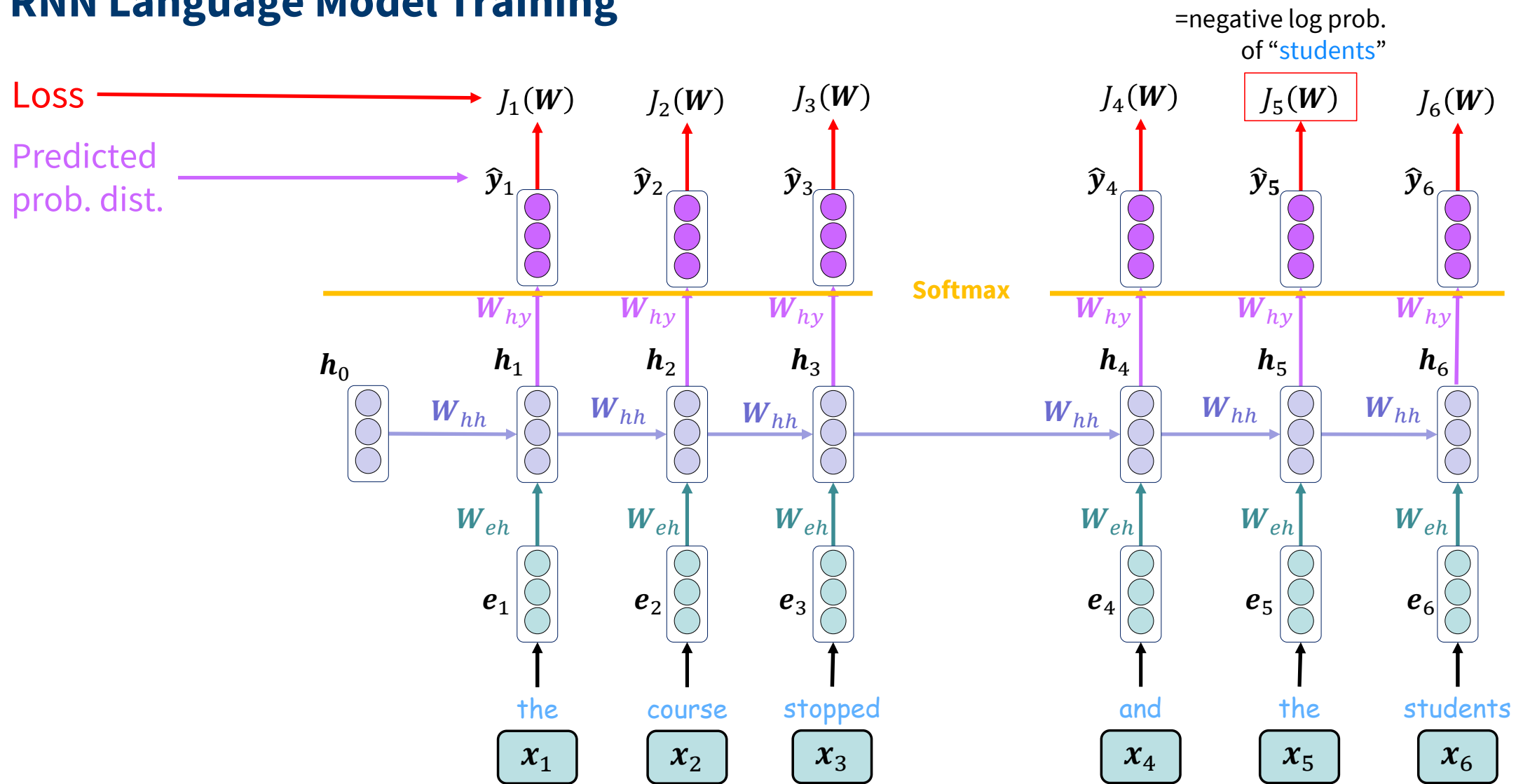
RNN Language Model Training



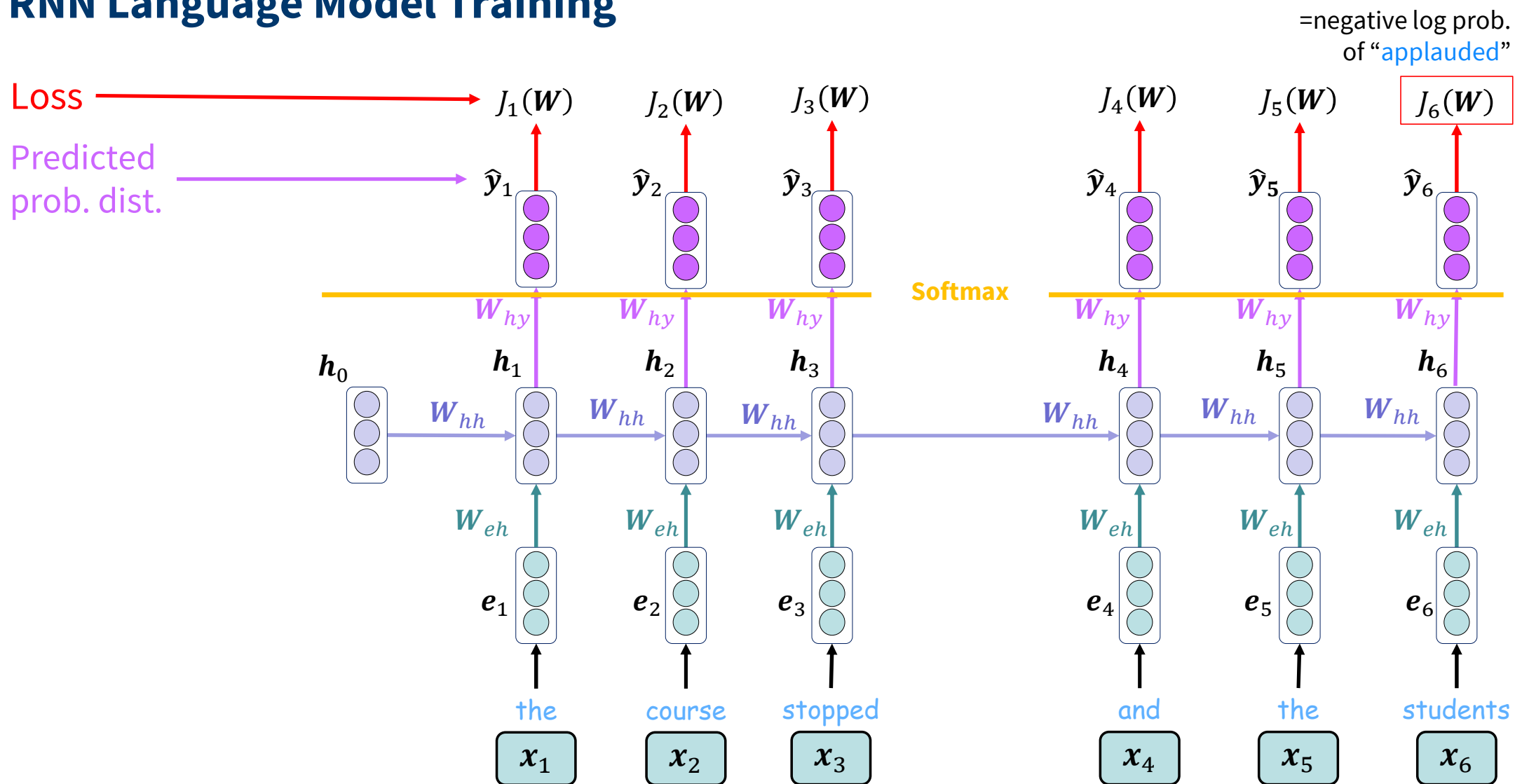
RNN Language Model Training



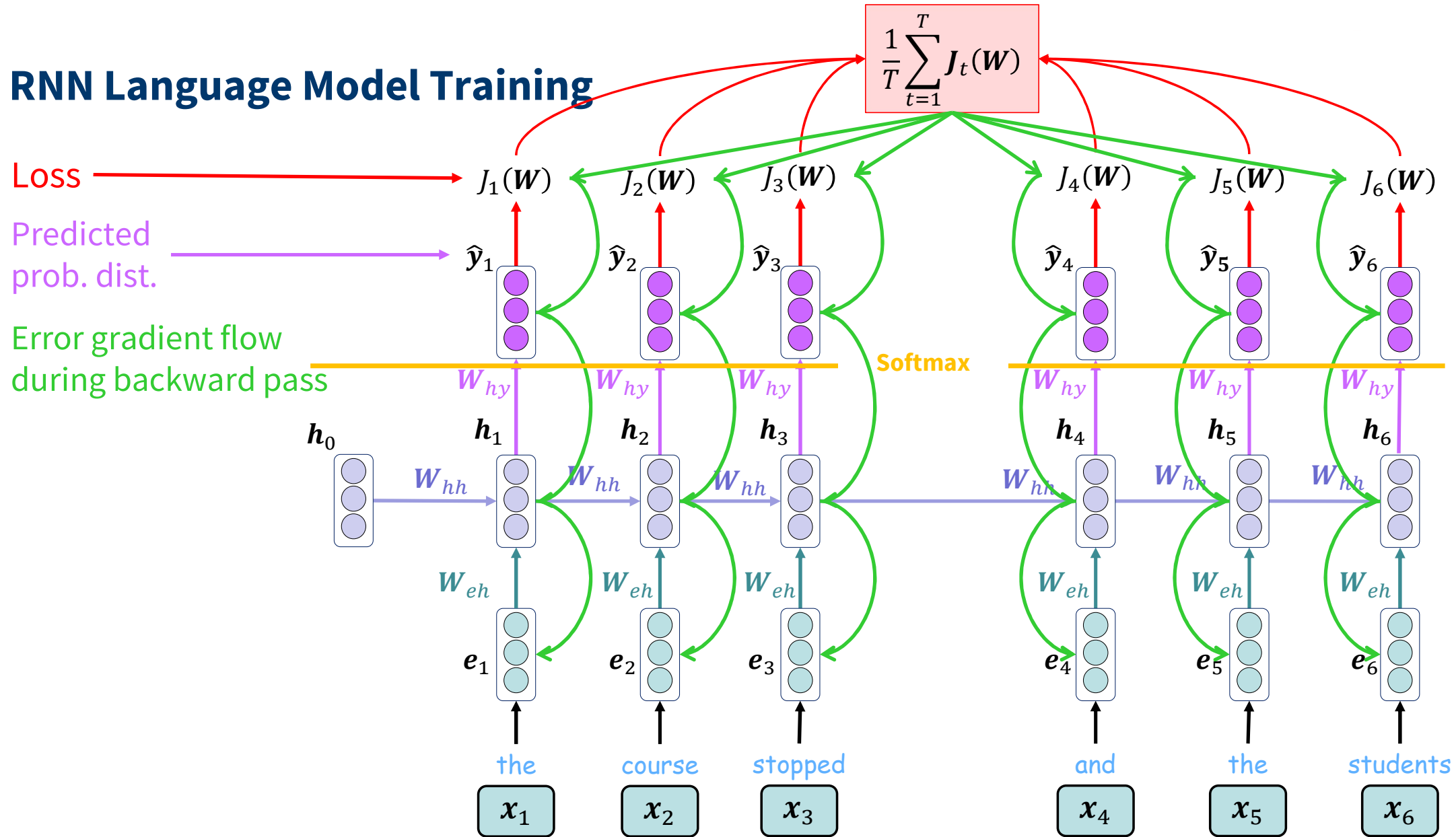
RNN Language Model Training



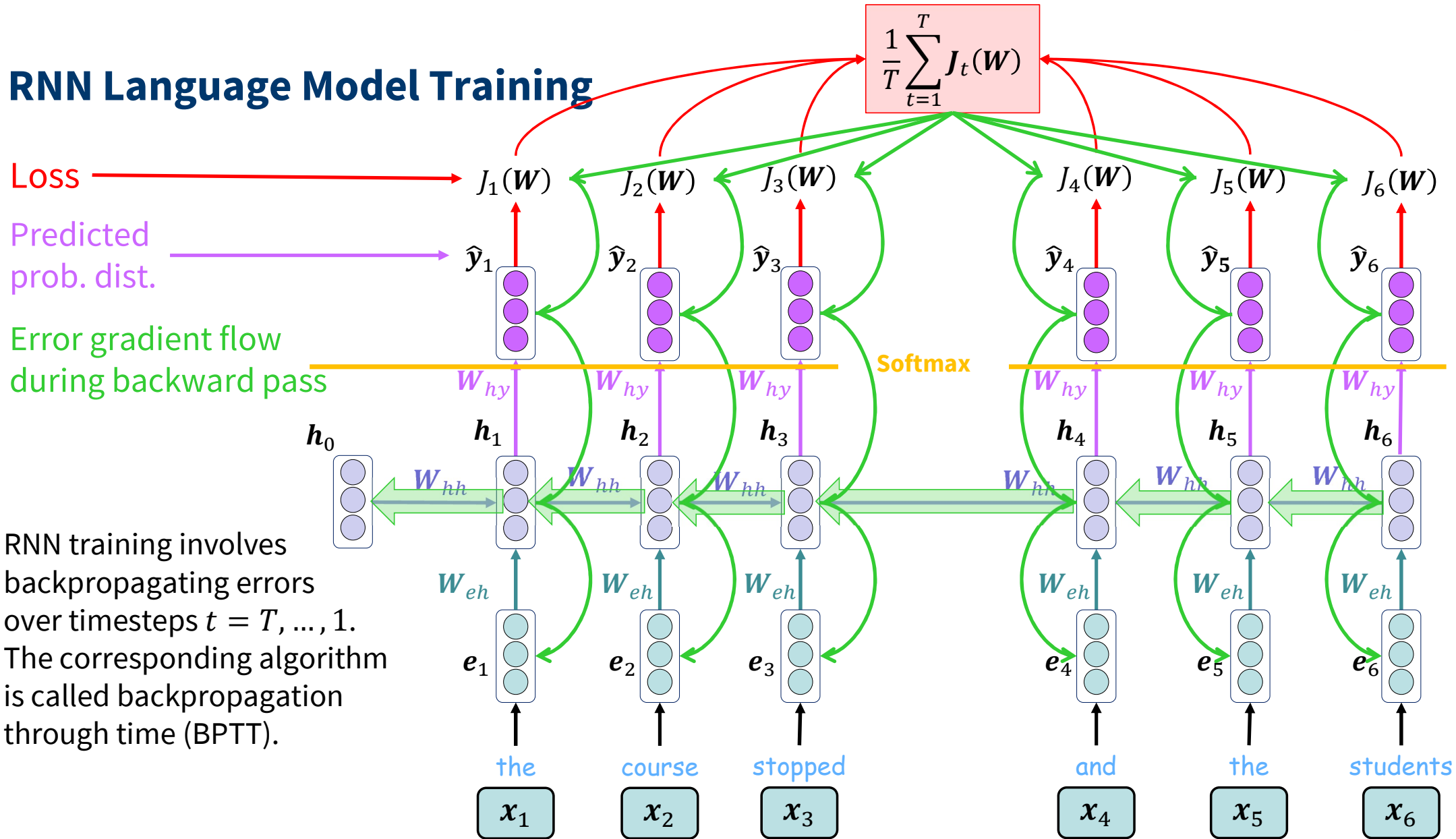
RNN Language Model Training



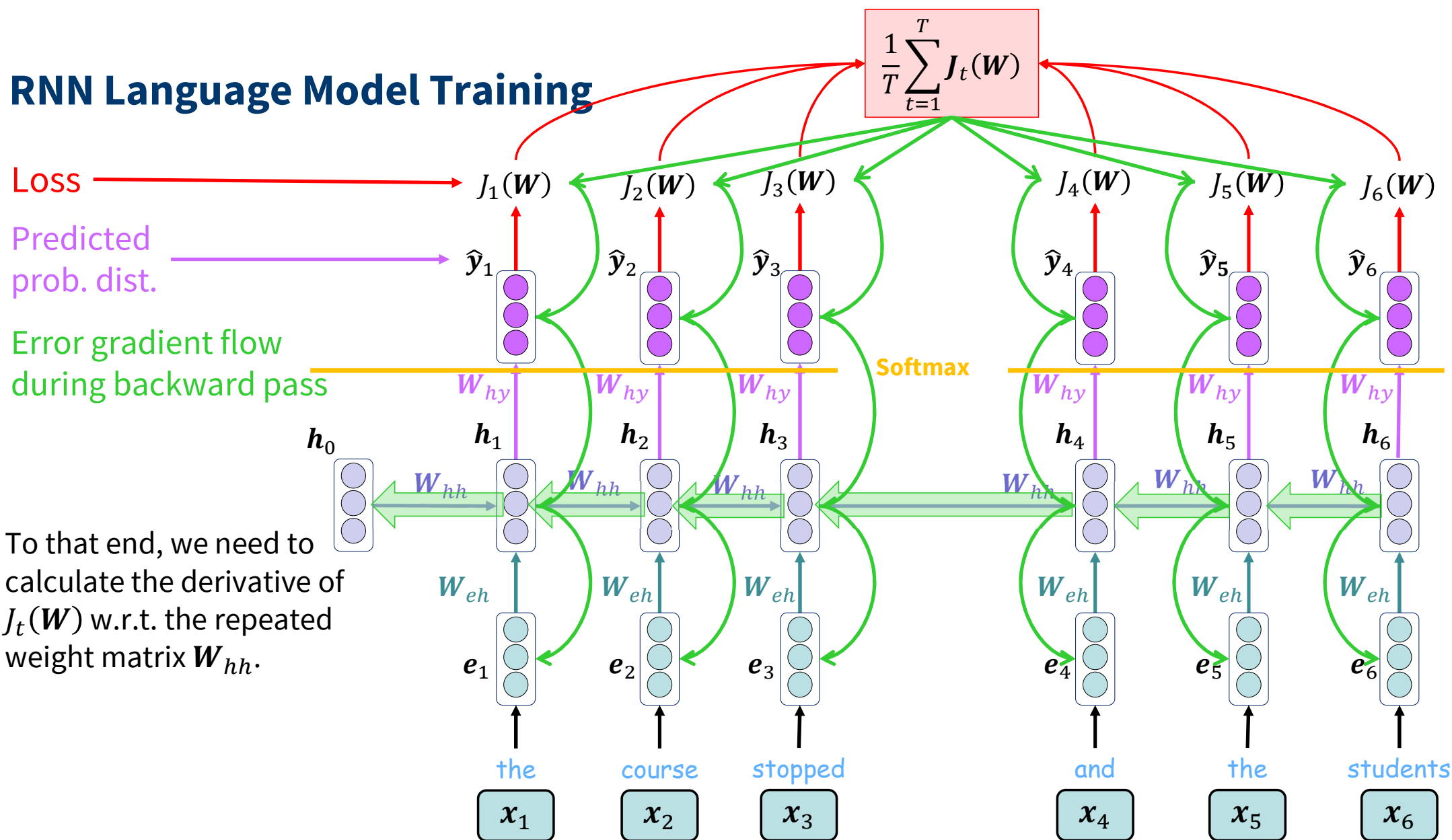
RNN Language Model Training



RNN Language Model Training



RNN Language Model Training

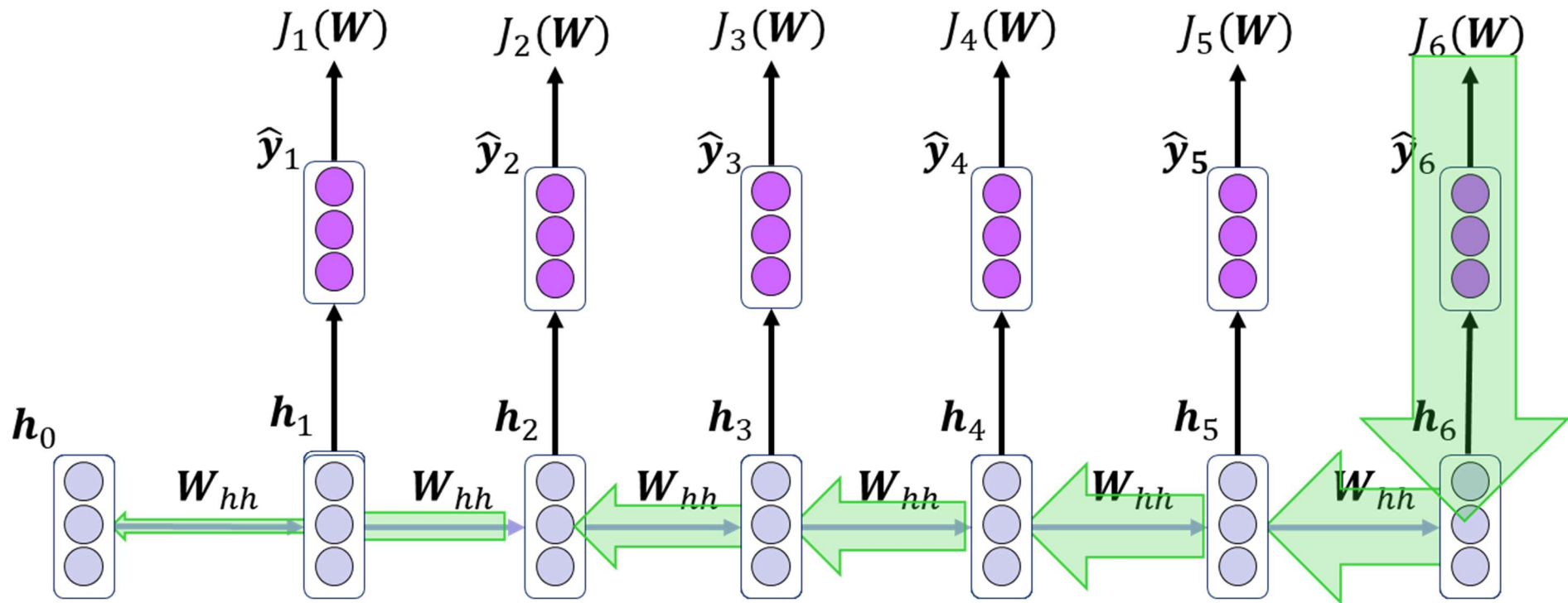


Backpropagation Through Time

- **An application of backpropagation to latent variable sequence models**
- **Involves long chains of derivatives**
- **In practice, BPTT is typically truncated for computational reasons**
 - Do not perform calculations all the way back to $\mathbf{h}_0$
 - Terminate the chain of computations at some – arbitrarily selected – threshold
 - See Tallec and Ollivier (2017) for details
- **Especially long sequences lead to large powers of $\mathbf{W}_{hh}$**
 - Weights smaller than 1 led to the problem of **gradient vanishing**
 - Weights larger than 1 led to the problem of **gradient explosion**
 - See Pascanu et al. (2013) for a proof

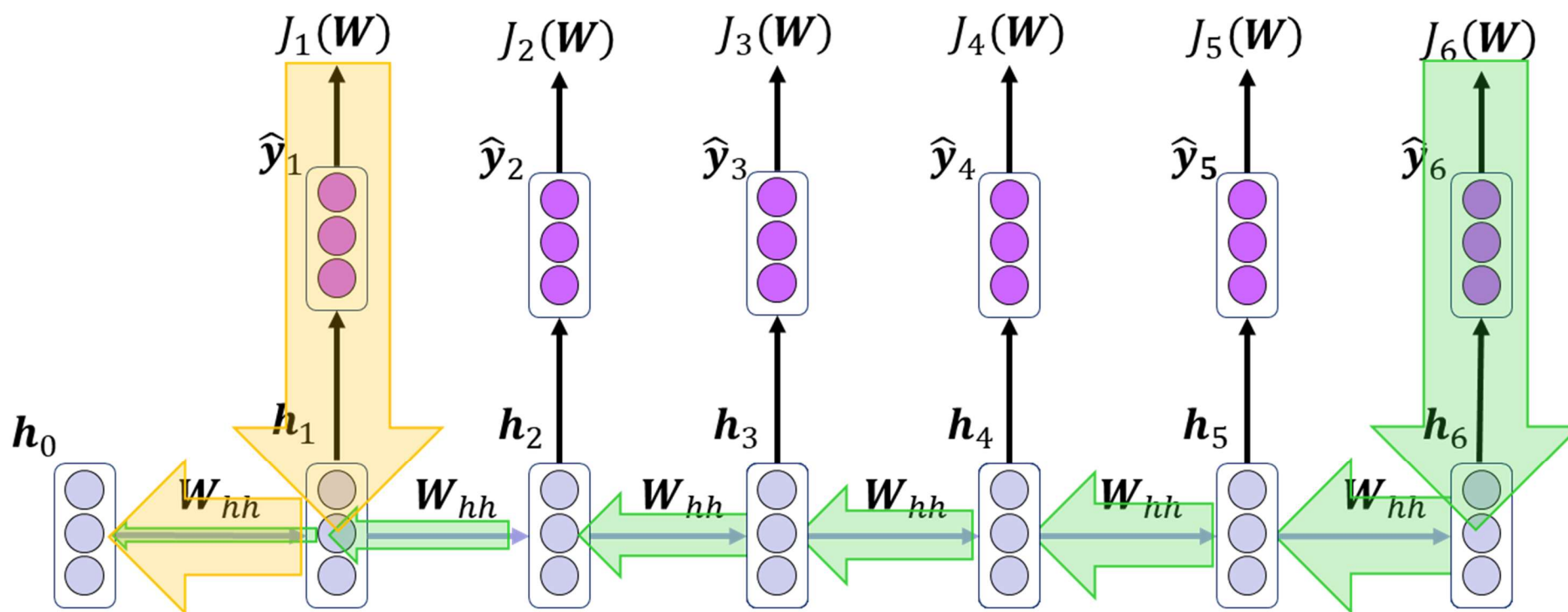
$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{j=1}^t (\mathbf{W}_{hh}^T)^{t-j} \mathbf{h}_j$$

The Problem of Vanishing Gradients



The **gradient signal** from faraway is lost because of repeated multiplications of the weight matrix when applying the chain rule.

The Problem of Vanishing Gradients



The **gradient signal** from faraway is lost and outweighed by **close by gradient signal**. Thus, weight updates emphasize **near effects** while the network faces difficulties in learning **long-term dependencies**.

The Problem of Vanishing Gradients

- Can interpret gradient as a measure of how much the past affects the future
- If the gradient signal vanishes over longer distances t to $t + j$
- There is no way to determine whether
 - There is no dependency between step t and $t + j$ in the data
 - We have the wrong parameters to capture the dependency between t and $t + j$
- Language modeling example
 - Model needs to learn the dependency between “France” and “speak fluent”
 - Gradient vanishing prohibits learning this dependency
 - Rendering the model unable to predict dependencies of such length

I grew up in France where my father works as a teacher in primary school. My mom is a criminal lawyer. Just recently, I moved to Berlin for my undergraduate studies. German proves difficult but I speak fluent ???.



Another Language Modeling Example

Linzen et al. (2016)

- Syntactic recency suggests "is"
- Sequential recency suggests "are"
- Due to the vanishing gradient problem, RNNs are better at learning from sequential recency than syntactic recency



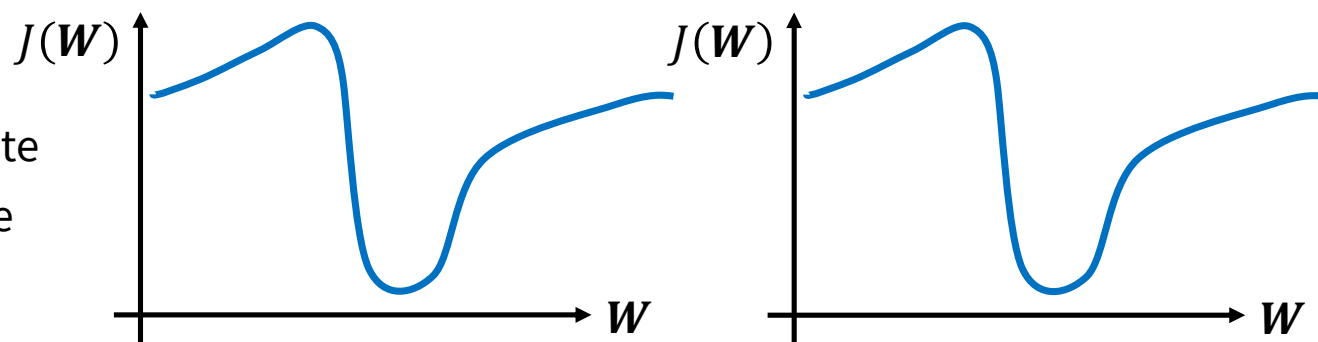
Problem of Exploding Gradients

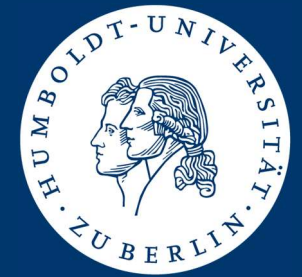
- Many weights larger than 1 lead to gradient explosion $\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{j=1}^t (\mathbf{W}_{hh}^T)^{t-j} \mathbf{h}_j$
- Gradient explosion is a problem because it can cause bad updates in the scope of gradient descent

$$\mathbf{W}_{hh} \leftarrow \mathbf{W}_{hh} - \eta \frac{\partial J(\mathbf{W}_{hh})}{\partial \mathbf{W}_{hh}} \quad \text{where} \quad \frac{\partial J(\mathbf{W}_{hh})}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \text{prod} \left(\frac{\partial \mathcal{L}(\mathbf{y}_t, \hat{\mathbf{y}}_t)}{\partial \hat{\mathbf{y}}_t}, \mathbf{W}_{hy}, \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} \right)$$

- Gradient clipping as solution

- “Cliff” is dangerous
- Steep gradient, large step, bad update
- If norm of the gradient exceeds some threshold, scale it down before applying SGD update





Gated RNNs for the rescue

Gated recurrent units (GRU), long short-term memory (LSTM), and extensions

Vanishing Gradient Problem Revisited

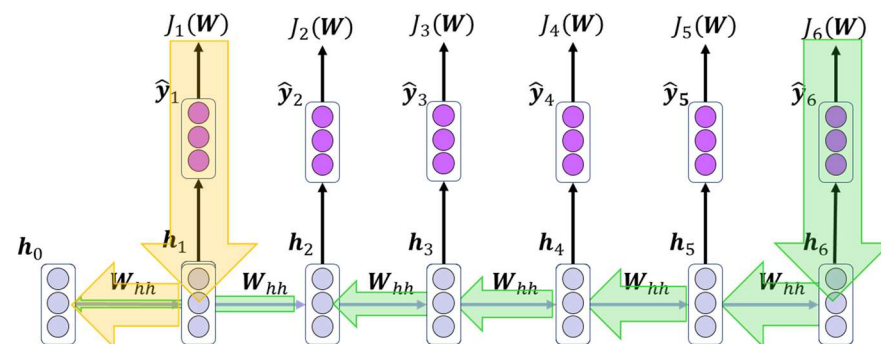
■ Multiplying many small numbers together in BPTT causes gradient vanishing

■ Gradient vanishing causes problems

- Difficult to learn long-term dependencies
- Updates of the hidden state emphasize recent information

■ Remedies

- [Weight initialization](#)
- [Use RELU activation function](#)
- Gated cells
 - NN needs to learn to preserve relevant information over many time steps
 - NN needs to learn to skip over irrelevant recent information



$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{j=1}^t (\mathbf{W}_{hh}^T)^{t-j} \mathbf{h}_j$$

I grew up in France where my father works as a teacher in primary school. My mom is a criminal lawyer. Just recently, I moved to Berlin for my undergraduate studies. German proves difficult but I speak fluent ???.



Vanishing Gradient Problem Revisited

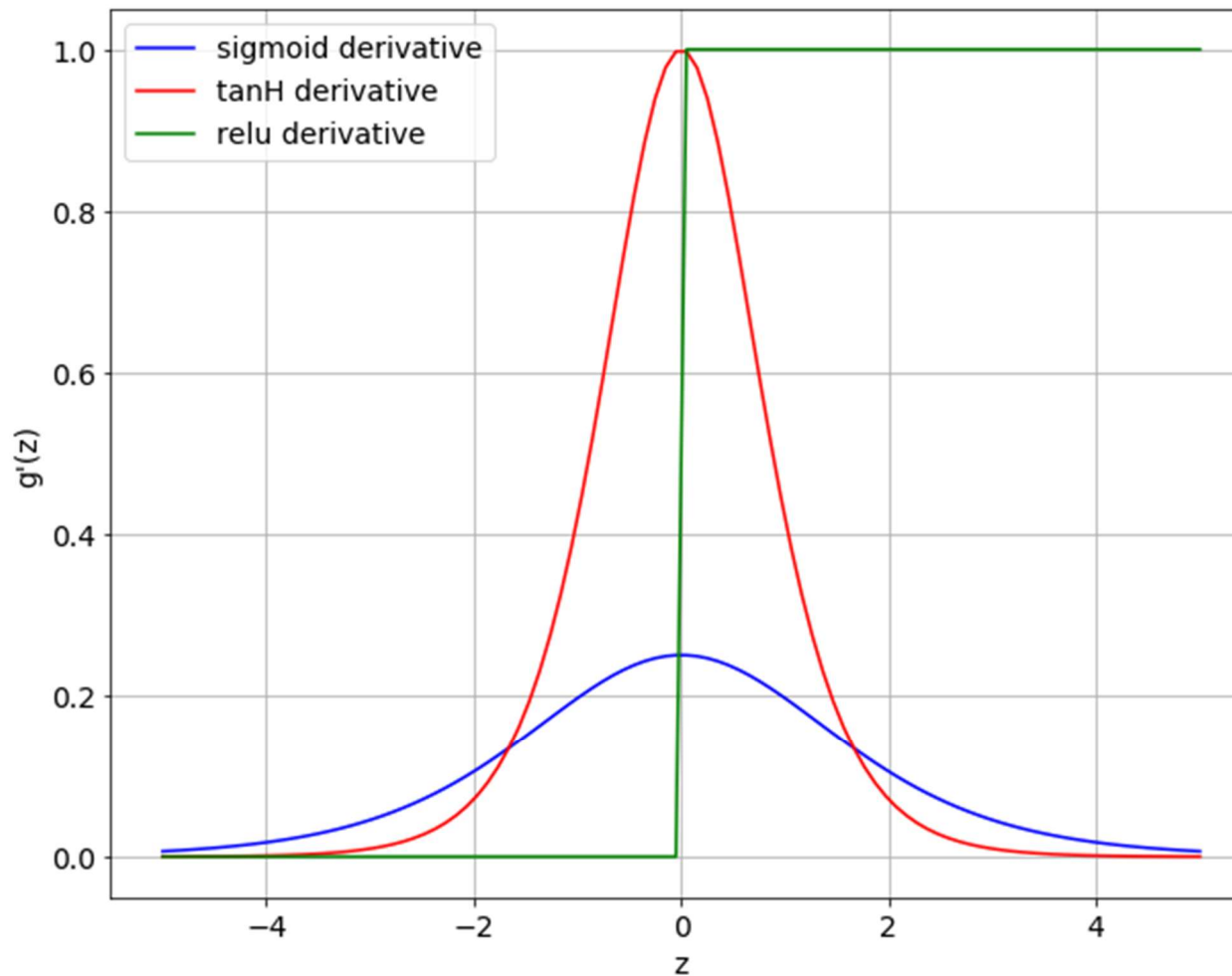
Weight initialization

- Initialize weights to identity matrix
- Initialize biases to zero
- This helps to prevent weights from shrinking to zero (too quickly)

$$I_n = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{pmatrix}$$

Vanishing Gradient Problem Revisited

RELU prevents g' from shrinking the gradients when $z > 0$



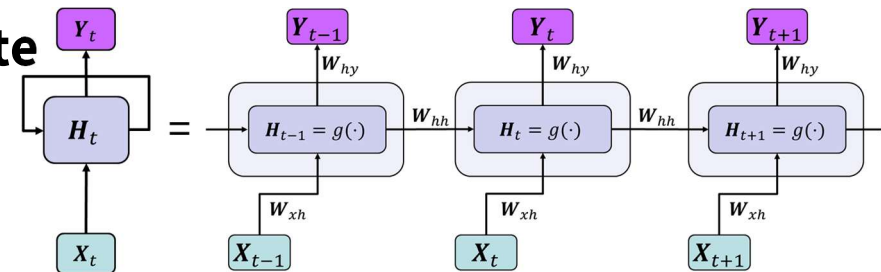
Gated Cells

Use a more complex recurrent unit

- Difficult for RNN to learn preserve information over many timesteps

- Because RNNs constantly rewrite the hidden state

$$H_t = g(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$



- Gated cells store a separate memory and control updates of that memory

- Memorize new information selectively
- Learn to forget (outdated) information

- Two standard designs of more complex recurrent units

- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit (GRU)

Gated Recurrent Units (GRU)¶

Chung et al. (2014)

■ Gating the hidden state

- Dedicated mechanism to decide when to update or reset the hidden state
- Learn this mechanism as part of training

■ What actually is meant by “gating”?

Gated Recurrent Units (GRU)

RNNs update their hidden state in every step

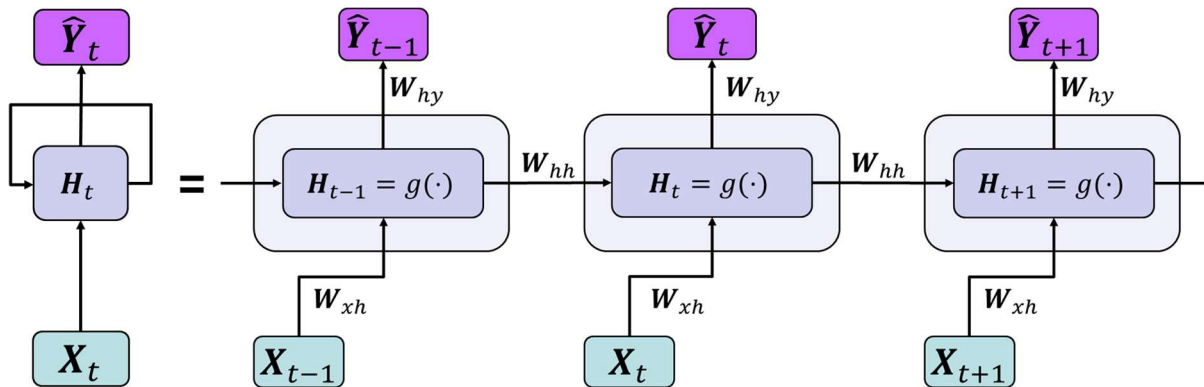
■ Gating the hidden state

- Dedicated mechanism to decide when to update or reset the hidden state
- Learn this mechanism as part of training

■ What actually is meant by “gating”?

$$\hat{Y}_t = H_t W_{hy} + b_y$$

$$H_t = g(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$



$$\begin{aligned} X_t &\in \mathbb{R}^{n \times m} \\ W_{xh} &\in \mathbb{R}^{m \times h} \\ b_h &\in \mathbb{R}^{1 \times h} \\ H_t &\in \mathbb{R}^{n \times h} \\ W_{hh} &\in \mathbb{R}^{h \times h} \\ W_{hy} &\in \mathbb{R}^{h \times c} \\ b_y &\in \mathbb{R}^{1 \times c} \end{aligned}$$

Gated Recurrent Units (GRU)

A dedicated matrix or gate could moderate these updates

■ Gating the hidden state

- Dedicated mechanism to decide when to update or reset the hidden state
- Learn this mechanism as part of training

■ What actually is meant by “gating”?

$$\hat{Y}_t = H_t W_{hy} + b_y$$

$$H_t = g(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

$$H_t = \begin{bmatrix} g(z_{11}) & \cdots & g(z_{1h}) \\ \vdots & \ddots & \vdots \\ g(z_{1n}) & \cdots & g(z_{nh}) \end{bmatrix} \odot \begin{bmatrix} u_{11} & \cdots & u_{1h} \\ \vdots & \ddots & \vdots \\ u_{1n} & \cdots & u_{nh} \end{bmatrix} = U_t$$

$$\begin{aligned} X_t &\in \mathbb{R}^{n \times m} \\ W_{xh} &\in \mathbb{R}^{m \times h} \\ b_h &\in \mathbb{R}^{1 \times h} \\ H_t &\in \mathbb{R}^{n \times h} \\ W_{hh} &\in \mathbb{R}^{h \times h} \\ W_{hy} &\in \mathbb{R}^{h \times c} \\ b_y &\in \mathbb{R}^{1 \times c} \\ U_t &\in \mathbb{R}^{n \times h} \end{aligned}$$

Gated Recurrent Units (GRU)

A dedicated matrix or gate could moderate these updates

■ Gating the hidden state

- Dedicated mechanism to decide when to update or reset the hidden state
- Learn this mechanism as part of training

■ What actually is meant by “gating”?

$$\hat{Y}_t = H_t W_{hy} + b_y$$

$$H_t = g(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$

$$H_t = \begin{bmatrix} g(z_{11}) & \cdots & g(z_{1h}) \\ \vdots & \ddots & \vdots \\ g(z_{1n}) & \cdots & g(z_{nh}) \end{bmatrix} \odot \begin{bmatrix} \sigma(u_{11}) & \cdots & \sigma(u_{1h}) \\ \vdots & \ddots & \vdots \\ \sigma(u_{1n}) & \cdots & \sigma(u_{nh}) \end{bmatrix} = U_t$$

$$\begin{aligned} X_t &\in \mathbb{R}^{n \times m} \\ W_{xh} &\in \mathbb{R}^{m \times h} \\ b_h &\in \mathbb{R}^{1 \times h} \\ H_t &\in \mathbb{R}^{n \times h} \\ W_{hh} &\in \mathbb{R}^{h \times h} \\ W_{hy} &\in \mathbb{R}^{h \times c} \\ b_y &\in \mathbb{R}^{1 \times c} \\ U_t &\in \mathbb{R}^{n \times h} \end{aligned}$$

Gated Recurrent Units (GRU)

Design ideas

■ Gating the hidden state

- Dedicated mechanism to decide when to update or reset the hidden state
- Learn this mechanism as part of training

■ What actually is meant by “gating”?

- Introduce a matrix of the same dimension as the hidden state
- Ensure elements of this matrix are between zero and one
- Perform element-wise multiplication

■ GRUs introduce two gates

- Update gate → governs which information is added to the hidden state
- Reset gate → governs which information is removed from the hidden state

■ Hidden state represents the memory of the network (same as RNN)

Gated Recurrent Units (GRU)

How to learn the gating mechanism?

- A gate is a matrix
- Learning a gate equates to learning the elements of a matrix
- Similar to learning a weight matrix
- Design gates as neural network

$$U_t = g(X_t W_{xu} + H_{t-1} W_{hu} + b_u)$$

$$R_t = g(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$$

$$X_t \in \mathbb{R}^{n \times m}$$

$$W_{xh} \in \mathbb{R}^{m \times h}$$

$$b_h \in \mathbb{R}^{1 \times h}$$

$$H_t \in \mathbb{R}^{n \times h}$$

$$W_{hh} \in \mathbb{R}^{h \times h}$$

$$W_{hy} \in \mathbb{R}^{h \times c}$$

$$b_y \in \mathbb{R}^{1 \times c}$$

$$U_t \in \mathbb{R}^{n \times h}$$

$$b_u \in \mathbb{R}^{1 \times h}$$

$$W_{xu} \in \mathbb{R}^{m \times h}$$

$$W_{hu} \in \mathbb{R}^{h \times h}$$

$$R_t \in \mathbb{R}^{n \times h}$$

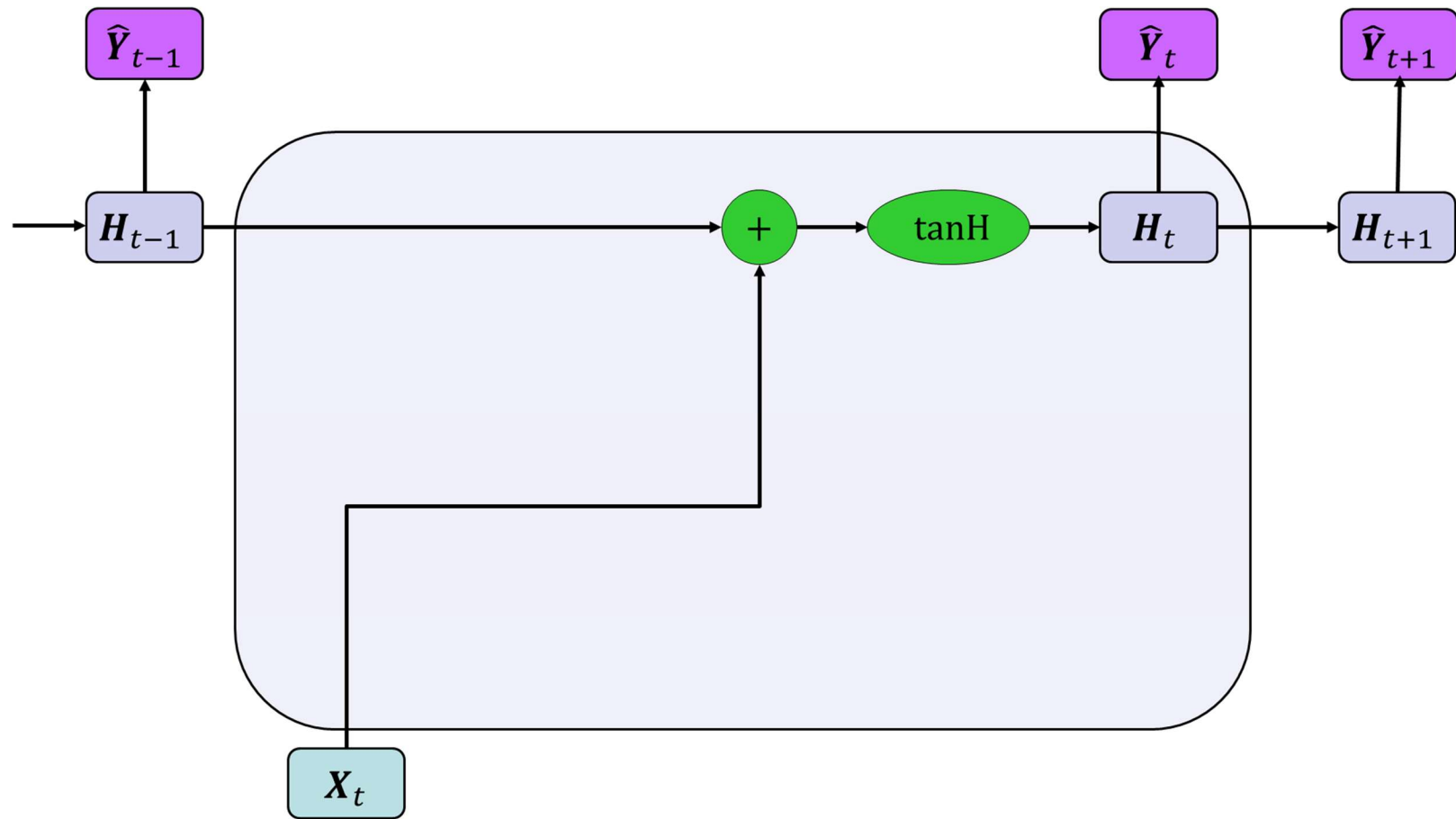
$$b_r \in \mathbb{R}^{1 \times h}$$

$$W_{xr} \in \mathbb{R}^{m \times h}$$

$$W_{hr} \in \mathbb{R}^{h \times h}$$

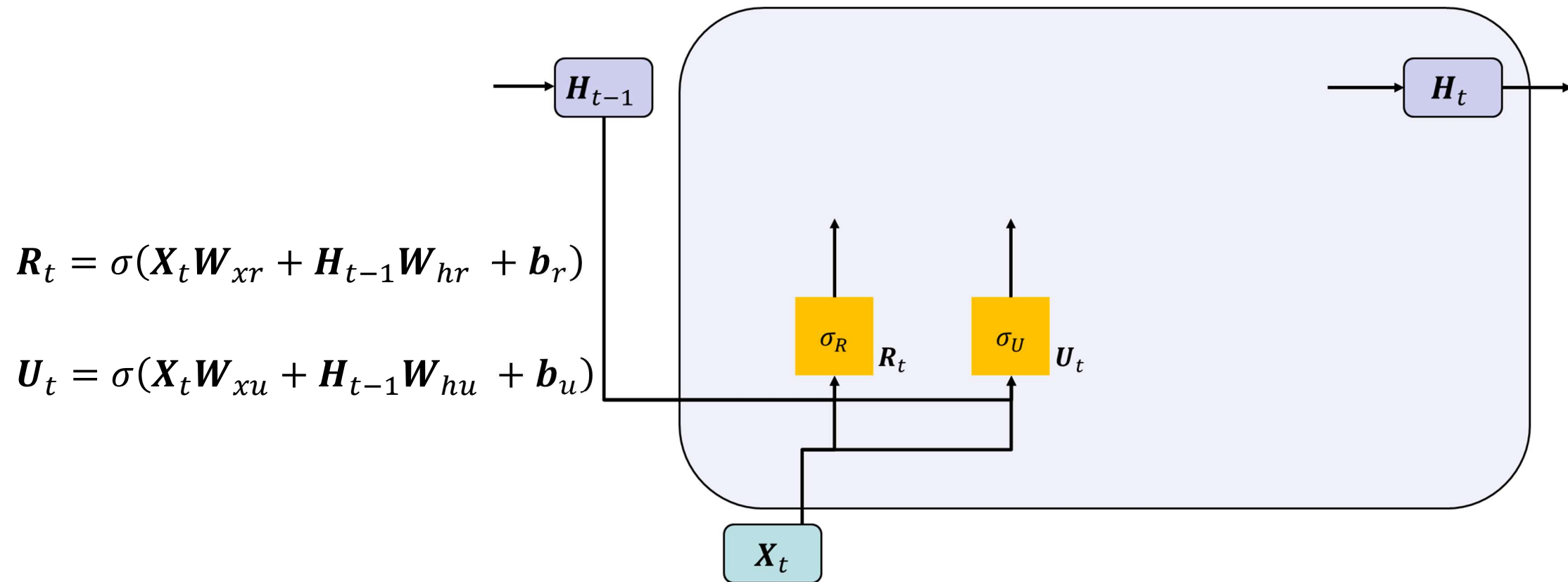
Gated Recurrent Units (GRU)¶

Graphical representation of RNN



Gated Recurrent Units (GRU)

Graphical representation of RNN with reset and update gate



Introducing a Reset Gate Into the GRU Cell

- Hidden state equation in RNN (note that $g(\cdot)$ is typically tanH)

$$\mathbf{H}_t = g(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h)$$

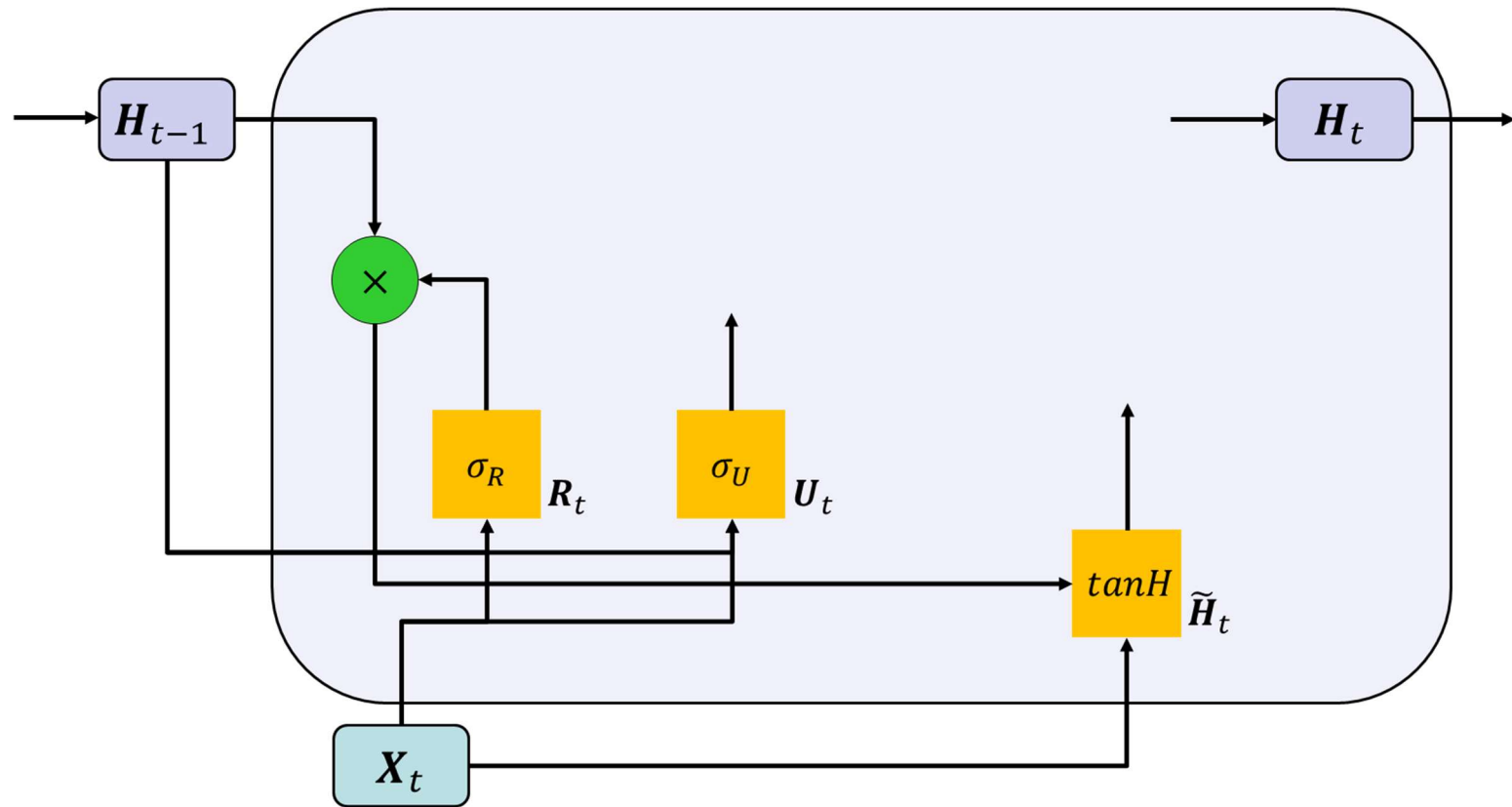
- GRU first generate a new candidate hidden state $\tilde{\mathbf{H}}_t$

$$\tilde{\mathbf{H}}_t = \text{tanH}(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h)$$

- Elements of $\mathbf{R}_t$ close to one:
- Elements of $\mathbf{R}_t$ close to zero:

Introducing a Reset Gate Into the GRU Cell

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h)$$



GRU Hidden State Update

- GRU first generate a new candidate hidden state $\hat{H}_t$

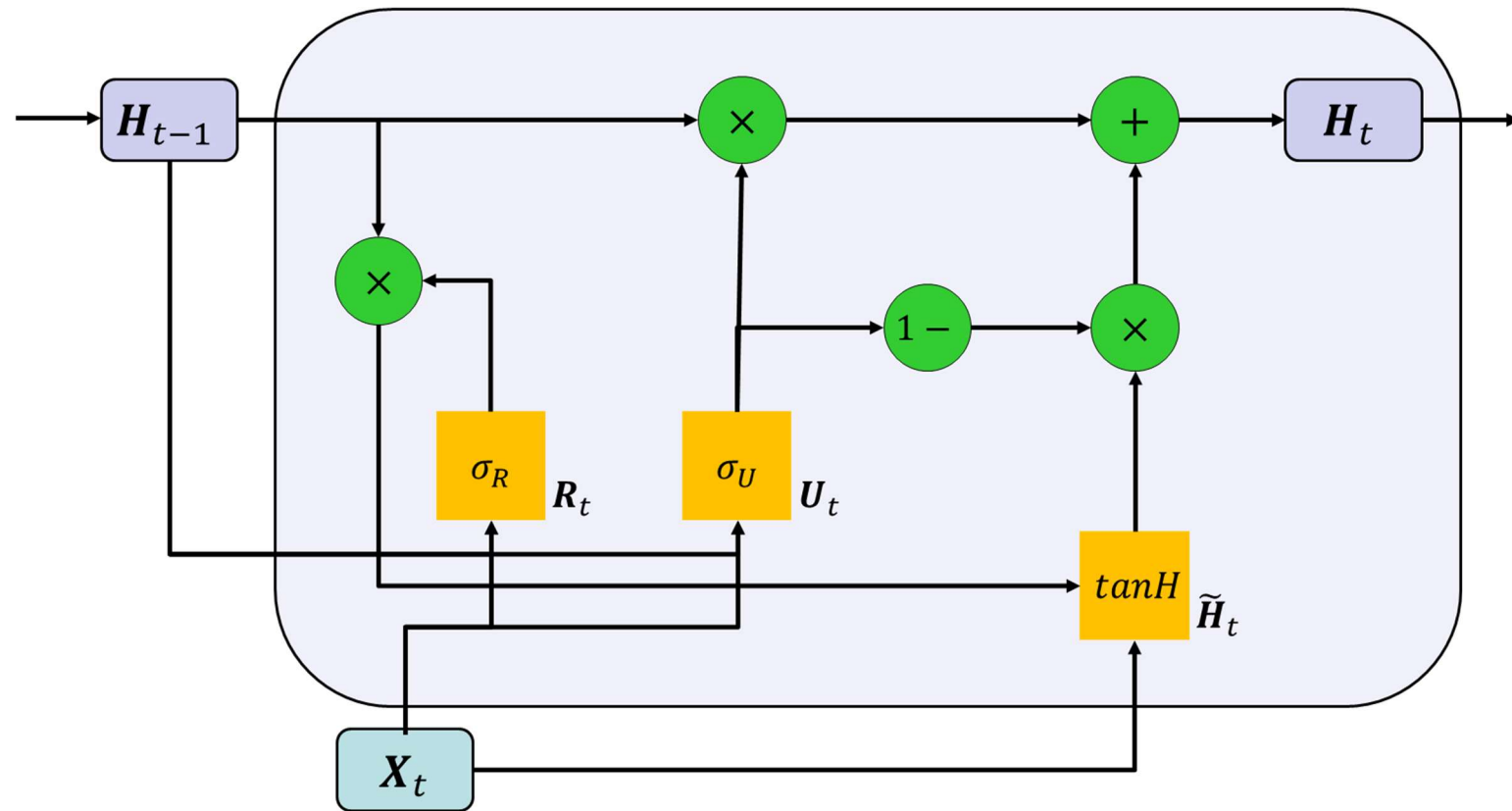
$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

- The candidate hidden state is then used to update the actual hidden state

$$H_t = U_t \odot H_{t-1} + (1 - U_t) \odot \tilde{H}_t$$

Full GRU Cell With Reset and Update Gate

$$\mathbf{H}_t = \mathbf{U}_t \odot \mathbf{H}_{t-1} + (\mathbf{1} - \mathbf{U}_t) \odot \tilde{\mathbf{H}}_t$$



Formal Summary of the GRU

■ **Reset Gate** $R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$

■ **Update Gate** $U_t = \sigma(X_t W_{xu} + H_{t-1} W_{hu} + b_u)$

■ **Candidate hidden state**

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

■ **Hidden state update**

$$H_t = U_t \odot H_{t-1} + (1 - U_t) \odot \tilde{H}_t$$

■ **Network output**

$$\hat{Y}_t = H_t W_{hy} + b_y$$

$$X_t \in \mathbb{R}^{n \times m}$$

$$W_{xh} \in \mathbb{R}^{m \times h}$$

$$b_h \in \mathbb{R}^{1 \times h}$$

$$H_t \in \mathbb{R}^{n \times h}$$

$$W_{hh} \in \mathbb{R}^{h \times h}$$

$$W_{hy} \in \mathbb{R}^{h \times c}$$

$$b_y \in \mathbb{R}^{1 \times c}$$

$$U_t \in \mathbb{R}^{n \times h}$$

$$b_u \in \mathbb{R}^{1 \times h}$$

$$W_{xu} \in \mathbb{R}^{m \times h}$$

$$W_{hu} \in \mathbb{R}^{h \times h}$$

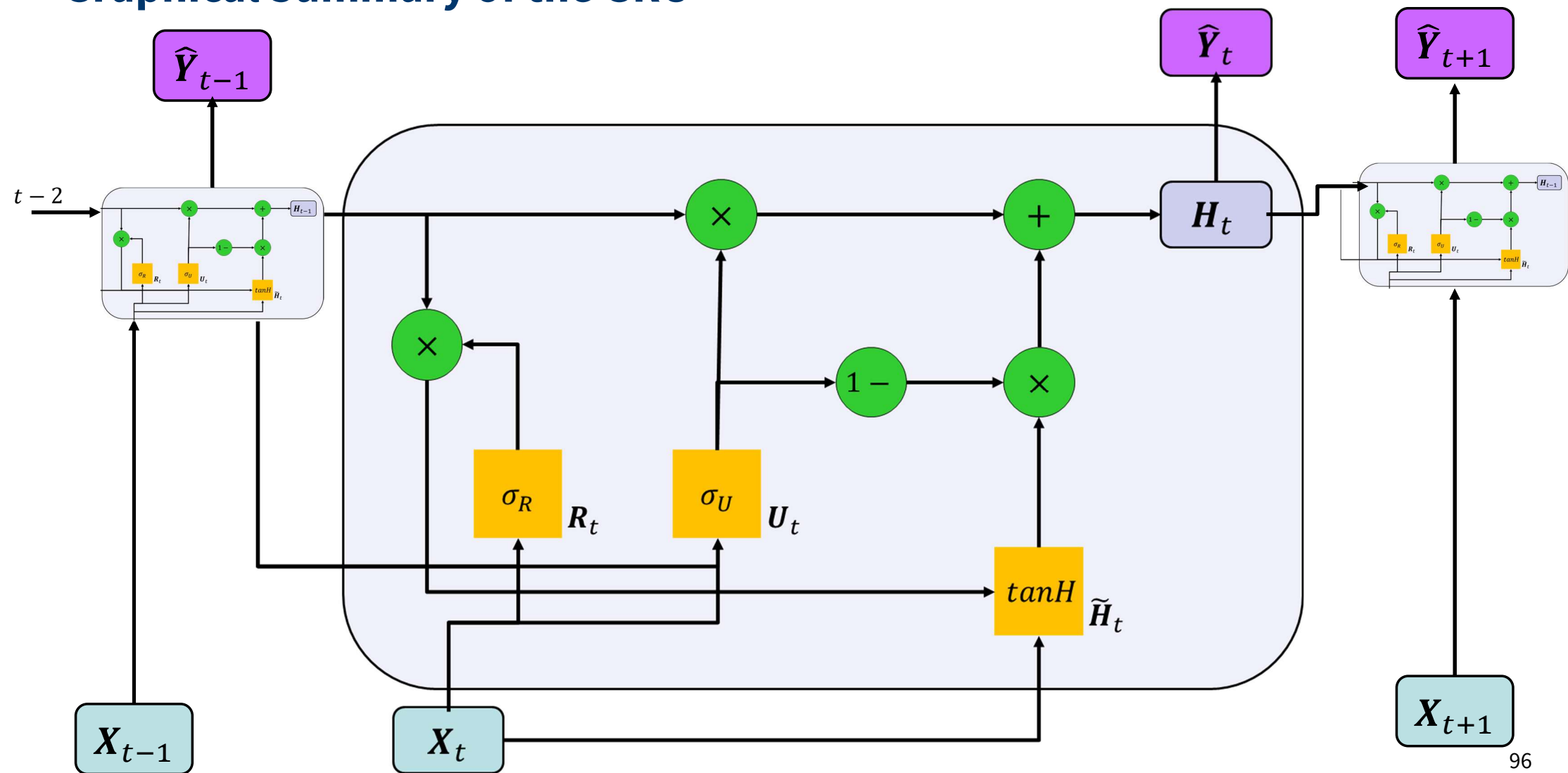
$$R_t \in \mathbb{R}^{n \times h}$$

$$b_r \in \mathbb{R}^{1 \times h}$$

$$W_{xr} \in \mathbb{R}^{m \times h}$$

$$W_{hr} \in \mathbb{R}^{h \times h}$$

Graphical Summary of the GRU



Long Short-Term Memory

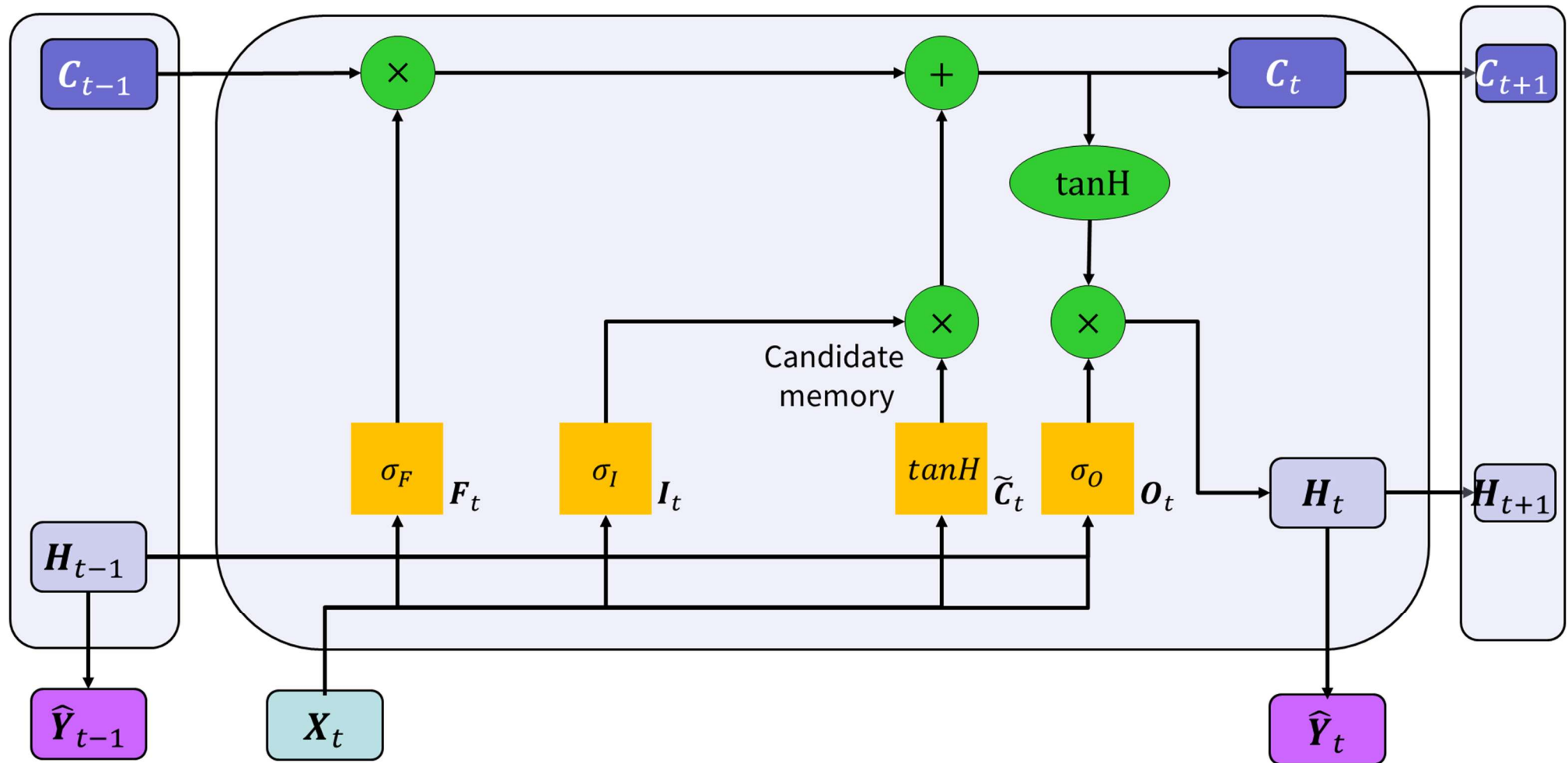
Hochreiter, Schmidhuber (1997)

- **Related approach to address long-term information preservation and short-term input skipping in latent variable models and solve the vanishing gradient problem**
- **Introduce an **additional cell memory**, which stores long-term information**
- **LSTM can erase, read, and write the memory via three gates**
 - Input gate
 - Output gate
 - Forget gate

} All designed as neural networks like with GRUs
- **Gates are dynamic in that their value is computed based on the current context, t**
 - Gates is closed $\rightarrow$ all matrix elements close to zero
 - Gates is open $\rightarrow$ all matrix elements close to one
 - Somewhere in between

Long Short-Term Memory

Visual representation of LSTM cell



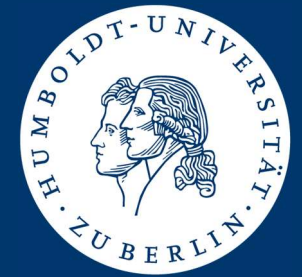
Long Short-Term Memory

Formal representation of LSTM cell

- **Forget gate** $F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$ Controls what is kept vs. forgotten from previous memory state
- **Input gate** $I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$ Controls what part of candidate memory is written to memory
- **Output gate** $O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)$ Controls what part of cell memory enters hidden state
- **New candidate memory** $\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$
- **Memory update** $C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$ Erase some part of memory and write some new content
- **Hidden state** $H_t = O_t \odot \tanh(C_t)$ Read some part from memory

Further Information on GRUs / LSTMs

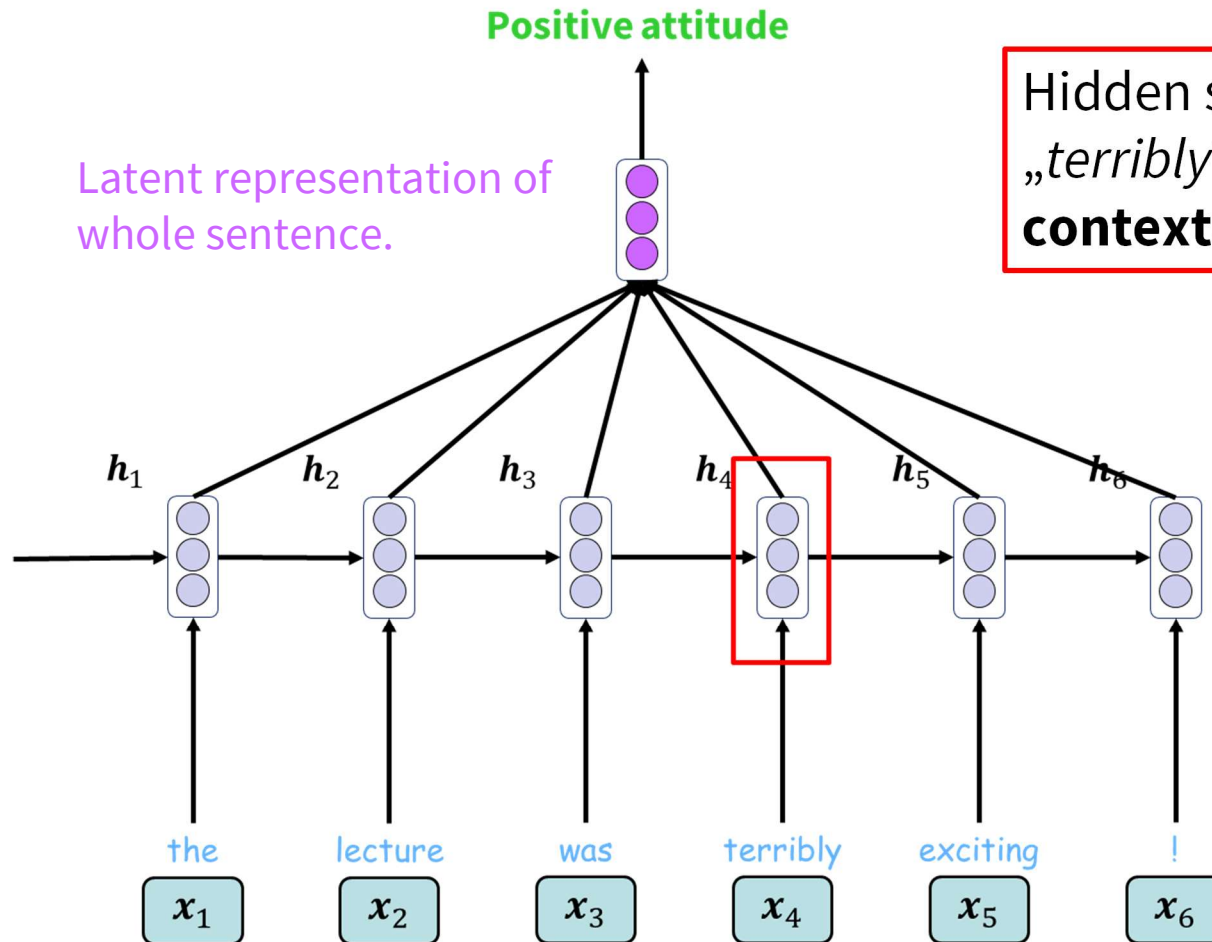
- Dive into Deep Learning Chapter 8 (foundation of this lecture)
- Famous blog on LSTMs: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- Critical perspective on LSTMs, suggesting CNN (see later) as superior alternative: <https://towardsdatascience.com/the-fall-of-rnn-lstm-2d1594c74ce0>
- Animated RNN, LSTM and GRU <https://towardsdatascience.com/animated-rnn-lstm-and-gru-ef124d06cf45>
- Illustrated Guide to LSTM's and GRU's: A step by step explanation <https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21>
- Stock modeling example <https://towardsdatascience.com/predicting-stock-price-with-lstm-13af86a74944>
- And many more...



Beyond Gated Recurrent Networks

A glance at cutting-edge RNN architectures

Bidirectional Recurrent Neural Networks



The contextual representation is based on the left context.

What about the right context?

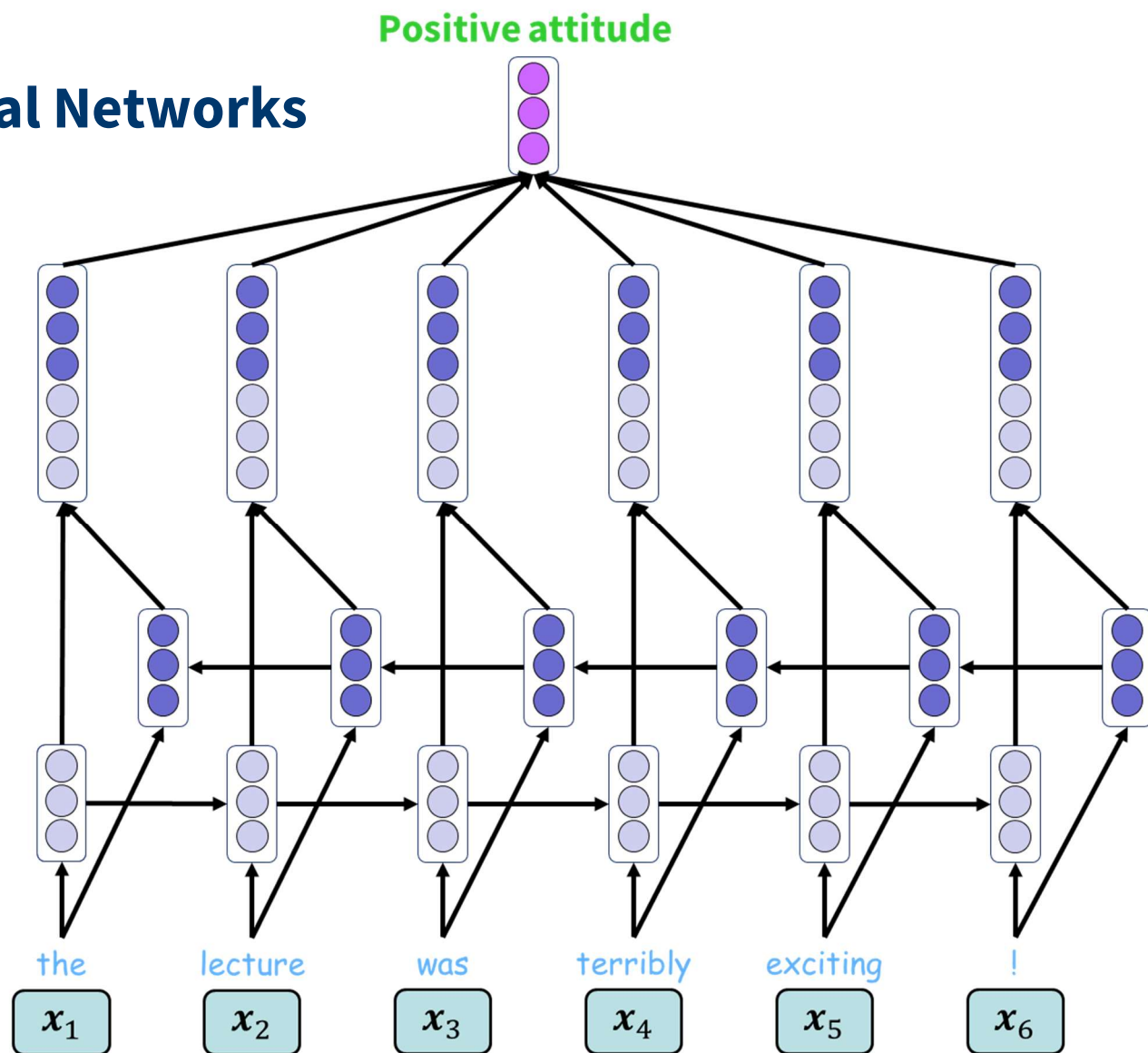
Here, the right context word „exciting“ shifts the polarity of the word „terribly“.

Bidirectional Recurrent Neural Networks

Concatenated hidden states

Backward RNN

Forward RNN



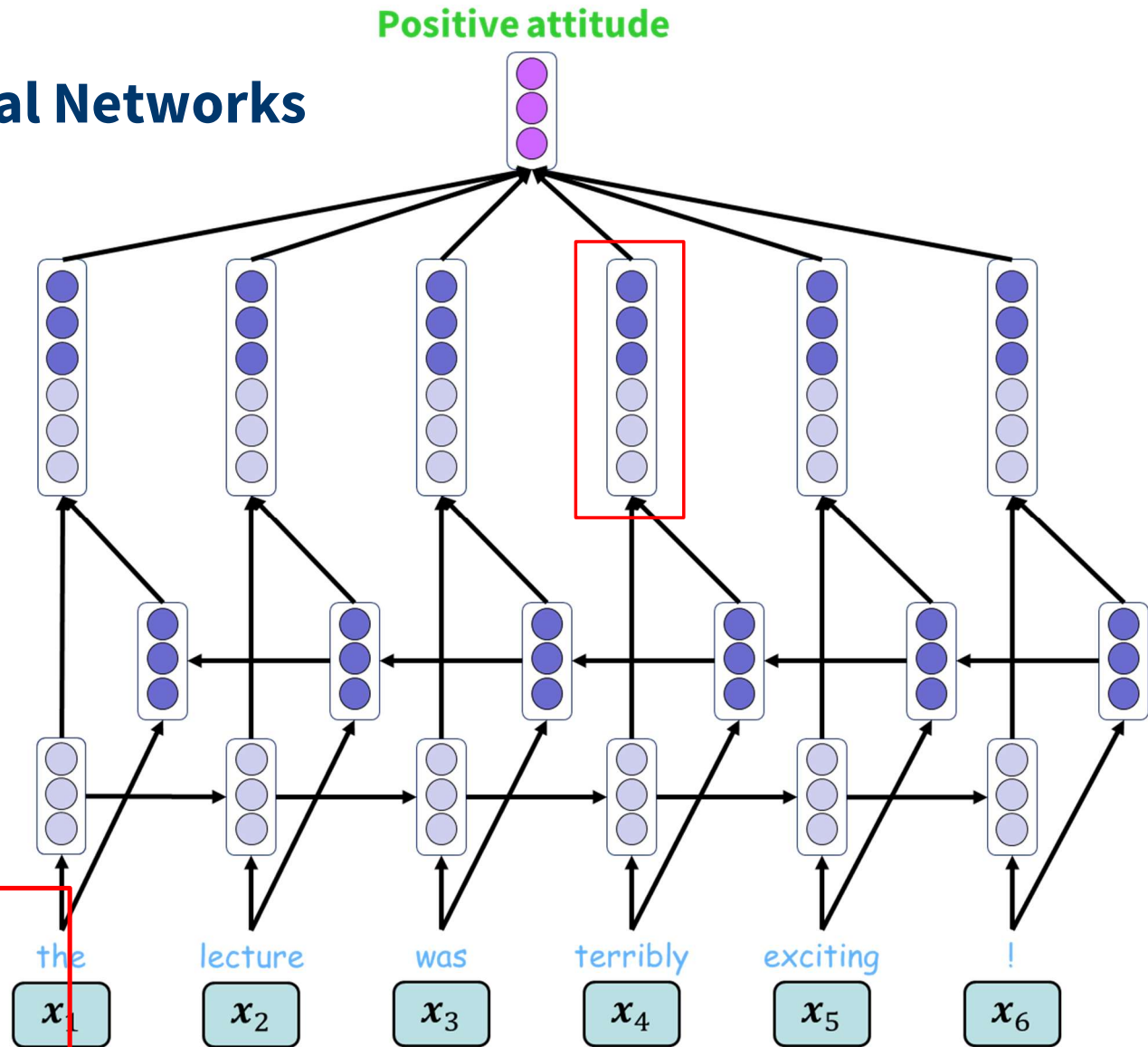
Bidirectional Recurrent Neural Networks

Concatenated hidden states

Backward RNN

Forward RNN

Contextual representation of word „*terribly*“ with both left and right context.



Bidirectional Recurrent Neural Networks

On one timestep t

Forward RNN

Backward RNN

Concatenated hidden states

Compute forward step of the RNN (vanilla, LSTM, GRU)

$$\vec{H}_t = RNN_{FW}(\vec{H}_{t-1}, X_t)$$

$$\overleftarrow{H}_t = RNN_{BW}(\overleftarrow{H}_{t+1}, X_t)$$

Two RNNs with separate weights

$$H_t = [\vec{H}_t, \overleftarrow{H}_t]$$

Hidden state of a bidirectional RNN, which we forward to the next part of the network.

Bidirectional Recurrent Neural Networks

Considerations on applicability

- **Bidirectional RNNs require access to the entire input sequence**

- Not applicable in language modeling where the goal is to predict the next word
- A language model has access to left context only
- Same in time series forecasting

- **Typically more powerful than unidirectional RNNs if requirement is met**

- **Example: BERT (Bidirectional Encoder Representations from Transformers)**

- Devlin et al. (2018)
- Powerful pretrained contextual representation system built on bidirectionality
- <https://ai.googleblog.com/2018/11/open-sourcing-bert-state-of-art-pre.html>
- <http://jalammar.github.io/a-visual-guide-to-using-bert-for-the-first-time/>

Multi-Layer Recurrent Neural Networks

- **RNNs exhibit temporal depth by definition**

- Enrolling computation over several time steps (i.e., sequence length)
- Example: Fischer & Krauss (2018) use a „deep“ network consisting of one LSTM cell

- **Can also obtain depth in a more conventional sense by stacking multiple layers of RNNs on top of each others**

- **May enable network to compute more complex representations**

- Lower network layers learn low-level features
- Deep layers learn more abstract, high-level features

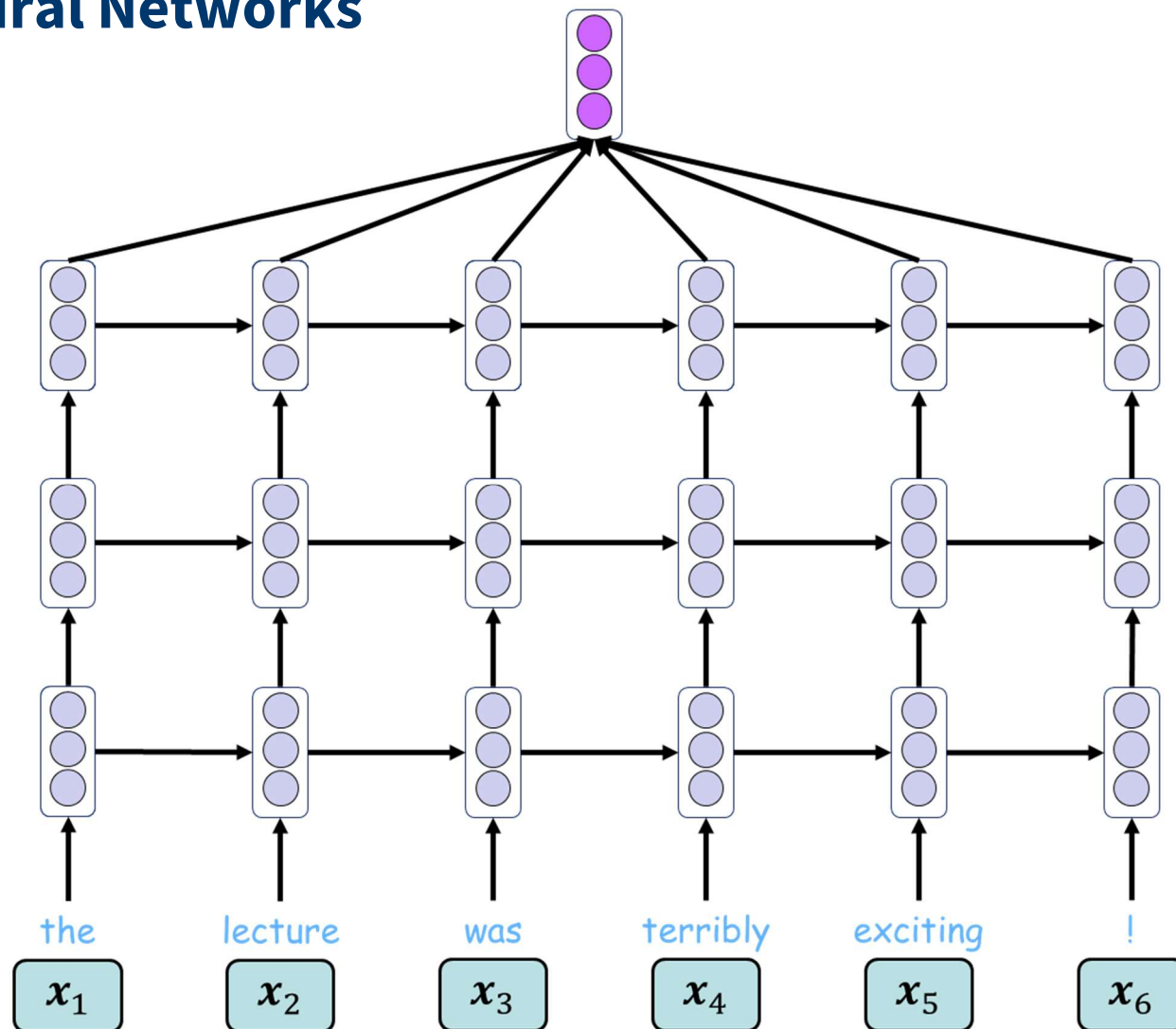
Multi-Layer Recurrent Neural Networks

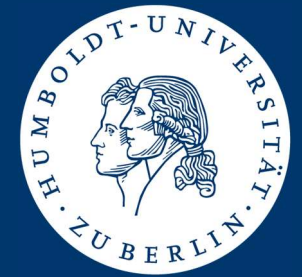
Positive attitude

Third layer RNN

Second layer RNN

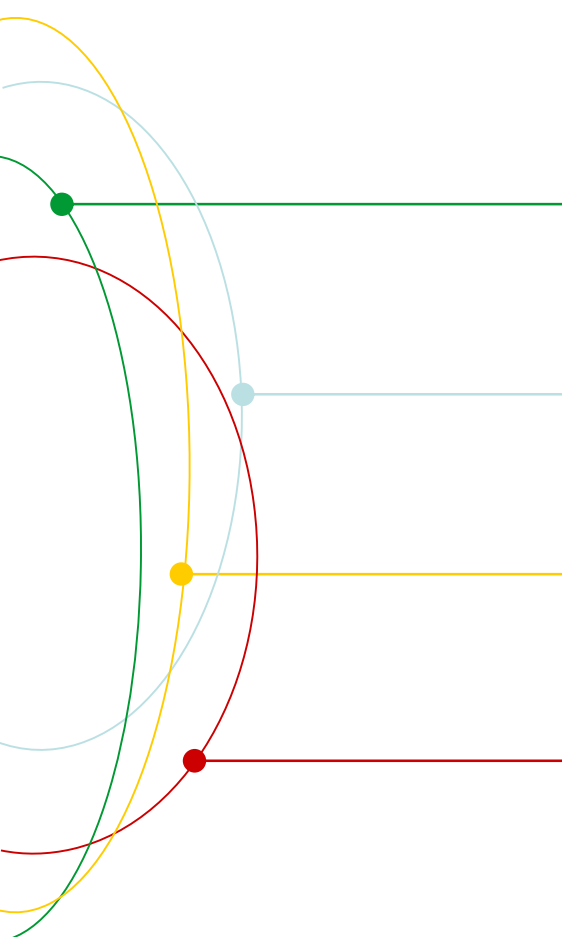
First layer RNN





Summary

Summary



Learning goals

- Training RNNs using BPTT
- Remedies to vanishing gradient problem



Findings

- Gated RNNs learn long-term dependencies
- Much more to discover (bidirectional, stacked, ...)



Homework

- Read Colah's famous post on LSTMs
- Read blog post on BBTT for LSTM/GRU



What next

- Evolution of NLP transfer learning
- Contextualization, finetuning, & attention

Credits & Disclaimer

- **Course materials have been inspired and draw upon numerous great resources available online**
- **The authors acknowledge the use of and wish to express their gratitude to the following sources**
 - Colah's blog, Understanding LSTM networks (<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>)
 - Zhang et al.: Dive into Deep Learning (<http://d2l.ai/index.html>)
 - MIT 6.S191: Deep Sequence Modeling (<http://introtodeeplearning.com/>)
 - Stanford University: Natural Language Processing with Deep Learning (<https://web.stanford.edu/class/cs224n/>)
 - Oxford University: Deep Natural Language Processing course (<https://github.com/oxford-cs-deepnlp-2017>)

Literature



- Chung J, Gulcehre C, Cho K, Bengio Y (2014): Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. CoRR arXiv:1412.3555v1
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *ArXiv preprint*, arXiv:1810.04805v2.
- Fischer T, Krauss C (2018): Deep learning with long short-term memory networks for financial market predictions. *EJOR* 270(2): 654-669
- Hochreiter S, Schmidhuber J (1997): Long Short-Term Memory. *Neural Computation* 9(8): 1735-1780
- Linzen et al. (2016) Assessing the Ability of LSTMs to Learn Syntax-Sensitive Dependencies. Archive preprint. <https://arxiv.org/pdf/1611.01368.pdf>
- R. Pascanu et al. (2013) On the difficulty of training recurrent neural networks. ICML. Available at <http://proceedings.mlr.press/v28/pascanu13.pdf>
- C. Tallec, Y. Ollivier (2017) Unbiasing Truncated Backpropagation Through. Archive preprint available at <https://arxiv.org/abs/1705.08209>

Credits & Disclaimer

- **Course materials have been inspired and draw upon numerous great resources available online**
- **The authors acknowledge the use of and wish to express their gratitude to the following sources**
 - Colah's blog, Understanding LSTM networks (<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>)
 - Zhang et al.: Dive into Deep Learning (<http://d2l.ai/index.html>)
 - MIT 6.S191: Deep Sequence Modeling (<http://introtodeeplearning.com/>)
 - Stanford University: Natural Language Processing with Deep Learning (<https://web.stanford.edu/class/cs224n/>)
 - Oxford University: Deep Natural Language Processing course (<https://github.com/oxford-cs-deepnlp-2017>)

Thank you for your attention!

Stefan Lessmann

Chair of Information Systems
School of Business and Economics
Humboldt-University of Berlin, Germany

Tel. +49.30.2093.5742

Fax. +49.30.2093.5741

stefan.lessmann@hu-berlin.de

<http://bit.ly/hu-wi>

www.hu-berlin.de

